প্র্যাকটিক্যাল গাইড

কন্টেক্সট ইঞ্জিনিয়ারিং হাতে-কলমে

একটি বাস্তব SaaS প্রোডাক্ট ও ই-কমার্স প্রজেক্টের মাধ্যমে শিখুন কীভাবে AI মডেলকে সঠিক তথ্য, সঠিক সময়ে দিতে হয়।

CS শিক্ষার্থী | জুনিয়র ডেভেলপার | RAG | LLM | প্রম্পট ডিজাইন

ইফতে সামউল ওহী

সফটওয়্যার ইঞ্জিনিয়ার ও কোর্স নির্মাতা

iftesamulohy.com

Licensed to sakilorb@gmail.com · Do not redistribute

কন্টেক্সট ইঞ্জিনিয়ারিং হাতে-কলমে

CS শিক্ষার্থী ও জুনিয়র ডেভেলপারদের জন্য একটি বাস্তবমুখী গাইড

লেখক — ইফতে সামউল ওহী

প্রথম সংস্করণ

iftesamulohy.com

Licensed to sakilorb@gmail.com · Do not redistribute

সূচিপত্র

কন্টেক্সট ইঞ্জিনিয়ারিং হাতে-কলমে
লেখক: ইফতে সামউল ওহী
iftesamulohy.com

২টি বাস্তব প্রজেক্ট	হাতে-কলমে কোড	সহজ বাংলা
TaskFlow ও ShopBondhu	কপি-পেস্ট করে চালানো যায়	উপমা ও চিত্র দিয়ে ব্যাখ্যা

শুরুর কথা

— ভূমিকা: কেন এই বই ৫

— এই বই কাদের জন্য ও কীভাবে পড়বেন ৬

— আমাদের দুই সঙ্গী প্রজেক্টের পরিচয় ৭

পর্ব ১ ভিত্তি গড়া

০১ কন্টেক্সট আসলে কী? ৯

১.১ একটা গল্প দিয়ে শুরু — নতুন কর্মীর প্রথম দিন ৯

১.২ LLM কীভাবে "ভাবে" — টোকেন ও context window ১০

১.৩ কন্টেক্সট = মডেলের কাজের পরিবেশ ১১

০২ Prompt Engineering বনাম Context Engineering ১৩

২.১ দুটোর আসল পার্থক্য ১৩

২.২ কখন কোনটা দরকার ১৪

২.৩ হাতে-কলমে: ভুল প্রম্পট vs সাজানো কন্টেক্সট ১৪

০৩ Context Window — মডেলের স্মৃতির সীমা ১৭

৩.১ টোকেন কী, কীভাবে গোনা হয় ১৭

৩.২ কেন সব তথ্য একসাথে দেওয়া যায় না ১৮

৩.৩ "Context Rot" — বেশি তথ্যের বিপদ ১৮

৩.৪ খরচ ও গতির হিসাব ১৯

পর্ব ২ কন্টেক্সটের উপাদান

০৪ System Prompt ও Instruction ডিজাইন ২২

৪.১ ভালো system prompt-এর শর্ত ২২

৪.২ ভূমিকা, নিয়ম ও সীমা ঠিক করা ২২

৪.৩ ShopBondhu সাপোর্ট বটের system prompt তৈরি ২৩

Licensed to sakilorb@gmail.com · Do not redistribute

০৫ **Few-shot Examples — উদাহরণ দিয়ে শেখানো** ২৬

৫.১ Zero-shot বনাম few-shot ২৬

৫.২ ভালো উদাহরণ বাছাইয়ের নিয়ম ২৬

৫.৩ TaskFlow: ইমেইল শ্রেণীবিন্যাস উদাহরণ ২৭

০৬ **Output Format নিয়ন্ত্রণ** ২৯

৬.১ JSON ও structured output আদায় করা ২৯

৬.২ XML ট্যাগ দিয়ে নির্ভরযোগ্যতা বাড়ানো ৩০

৬.৩ নিরাপদে পার্সিং ও ভ্যালিডেশন ৩০

পর্ব ৩ **বাইরের জগতের সাথে সংযোগ**

০৭ **RAG — প্রাসঙ্গিক ডেটা টেনে আনা** ৩৩

৭.১ RAG কী ও কেন দরকার ৩৩

৭.২ Embedding ও vector search সহজ ভাষায় ৩৩

৭.৩ Chunking কৌশল ৩৪

৭.৪ ShopBondhu প্রোডাক্ট ক্যাটালগ দিয়ে RAG ৩৫

০৮ **Conversation History ও Memory** ৩৭

৮.১ মাল্টিটার্ন কথোপকথন সামলানো ৩৭

৮.২ হিস্ট্রি ছোট রাখার কৌশল (summarization) ৩৮

৮.৩ দীর্ঘমেয়াদি memory ডিজাইন ৩৮

০৯ **Tools ও Function Calling** ৪০

৯.১ মডেলকে "হাত-পা" দেওয়া ৪০

৯.২ টুল সংজ্ঞা (schema) লেখা ৪০

৯.৩ ShopBondhu: order_status ও refund টুল ৪১

৯.৪ নিরাপত্তা ও সীমা নির্ধারণ ৪১

পর্ব ৪ **সব একসাথে — পেশাদারি কৌশল**

১০ **Context সাজানো ও অপটিমাইজেশন** ৪৫

১০.১ ক্রম (ordering) কেন গুরুত্বপূর্ণ ৪৫

১০.২ প্রাসঙ্গিকতা বনাম শব্দসংখ্যা ৪৫

১০.৩ টোকেন বাজেট সামলানো ৪৬

১১ **পূর্ণ কেস স্টাডি: ShopBondhu চ্যাটবট** ৪৮

১১.১ সমস্যা সংজ্ঞায়িত করা ৪৮

১১.২ কন্টেক্সট পাইপলাইন ডিজাইন ৪৮

Licensed to sakilorb@gmail.com · Do not redistribute

১১.৩ ধাপে ধাপে কোড ৪৯
১১.৪ সাধারণ বাগ ও সমাধান ৫০

১২ **সাধারণ ভুল ও সেরা অভ্যাস (চেকলিস্ট** ৫৩

পরিশিষ্ট

ক **Context Engineering চিটশিট** ৫৬

খ **শব্দকোষ (টার্মের বাংলা অর্থ)** ৫৭

গ **আরও শেখার রিসোর্স** ৫৯

ভূমিকা: কেন এই বই

গত কয়েক বছরে ChatGPT, Claude-এর মতো ভাষা মডেল আমাদের কাজের ধরন বদলে দিয়েছে। কিন্তু একটা মজার ব্যাপার লক্ষ্য করেছেন কি? একই মডেল ব্যবহার করে একজন ডেভেলপার দারুণ একটা AI অ্যাপ বানিয়ে ফেলেন, আর আরেকজনের অ্যাপ ভুলভাল উত্তর দেয়, এলোমেলো আচরণ করে। পার্থক্যটা কোথায়?

পার্থক্যটা মডেলে নয় — পার্থক্যটা **কন্টেক্সটে**। অর্থাৎ, মডেলকে কাজ দেওয়ার সময় আপনি তাকে কী তথ্য, কোন নির্দেশনা, আর কোন উদাহরণ দিচ্ছেন — সেটাই আসল খেলা। এই দক্ষতাকেই বলে **Context Engineering**, আর এটাই এই বইয়ের মূল বিষয়।

বাজারে প্রম্পট নিয়ে অনেক লেখা আছে, কিন্তু বাংলায় বাস্তব প্রজেক্ট ধরে ধরে context engineering শেখানো হয় — এমন কিছু খুব কম। আমি এই বইটা লিখেছি যাতে আপনি শুধু তত্ত্ব মুখস্থ না করে, নিজের হাতে একটা কাজ করা সিস্টেম দাঁড় করাতে পারেন।

এই বইয়ের প্রতিটা ধারণা আমরা দুটো বাস্তব প্রজেক্টের সাথে মিলিয়ে শিখব — একটা SaaS টুল আর একটা ই-কমার্স প্রজেক্ট। ফলে প্রতিটা কৌশল শুধু "জানা" থাকবে না, "ব্যবহার করতে পারা"-ও থাকবে।

এক কথায়

ভালো AI অ্যাপ মানে বড় মডেল নয় — ভালো AI অ্যাপ মানে ভালোভাবে সাজানো কন্টেক্সট। এই বই আপনাকে সেই সাজানোর শিল্পটাই শেখাবে।

Licensed to sakilorb@gmail.com · Do not redistribute

এই বই কাদের জন্য ও কীভাবে পড়বেন

কাদের জন্য

এই বই লেখা হয়েছে মূলত **কম্পিউটার সায়েন্সের শিক্ষার্থী** এবং **জুনিয়র ডেভেলপারদের** কথা মাথায় রেখে। আপনি যদি অন্তত একটা প্রোগ্রামিং ভাষা (বিশেষত Python) মোটামুটি বোঝেন এবং API কী জিনিস সে সম্পর্কে ধারণা রাখেন, তাহলেই যথেষ্ট। মেশিন লার্নিং বা নিউরাল নেটওয়ার্কের গভীর জ্ঞান লাগবে না — যেখানে দরকার, সহজ ভাষায় বুঝিয়ে দেব।

কীভাবে পড়বেন

তাড়াহুড়ো করবেন না। প্রতিটা অধ্যায়ে কোডের উদাহরণ আছে — পড়ার সময় শুধু চোখ বুলিয়ে না গিয়ে নিজে টাইপ করে চালিয়ে দেখুন। ভুল হলে ভয় পাবেন না; বরং ভুল থেকেই context engineering-এর আসল অনুভূতিটা আসে।

বইয়ের কাঠামো ধাপে ধাপে এগিয়েছে — সহজ ভিত্তি থেকে শুরু করে ধীরে ধীরে বাস্তব সিস্টেম পর্যন্ত। তাই অধ্যায়গুলো ক্রম অনুযায়ী পড়াই ভালো।

> **ছোট পরামর্শ**
>
> প্রতিটা অধ্যায়ের শেষে "কী শিখলেন" অংশটা আছে। একটা অধ্যায় শেষ করে যদি ওই পয়েন্টগুলো নিজের ভাষায় বলতে পারেন, তাহলে বুঝবেন ধারণাটা পাকা হয়েছে।

Licensed to sakilorb@gmail.com · Do not redistribute

আমাদের দুই সঙ্গী প্রজেক্ট

পুরো বইজুড়ে আমরা কাল্পনিক কিন্তু বাস্তবসম্মত দুটো প্রজেক্ট ব্যবহার করব। এদের সাথে আপনার পরিচয় করিয়ে দিই, কারণ প্রতিটা অধ্যায়ে এদের নাম বারবার আসবে।

TaskFlow — একটি SaaS প্রজেক্ট

টিমের কাজ গুছিয়ে রাখার একটা অনলাইন টুল, অনেকটা Trello বা Asana-র মতো — প্রজেক্টটাস্ক, ব্যবহারকারী আর কমেন্ট নিয়ে একটা পূর্ণ অ্যাপ্লিকেশন। এই বইয়ে আমরা গোটা অ্যাপটা বানাব না; আমরা ফোকাস করব এর **AI সহকারী** অংশে, যা চারটা কাজ করে:

• আসা টাস্ক স্বয়ংক্রিয়ভাবে শ্রেণীবদ্ধ করা • প্রজেক্টের অবস্থা সারসংক্ষেপ করা • অগ্রাধিকার পরামর্শ দেওয়া • "কোন প্রজেক্ট আটকে আছে?" ধরনের প্রশ্নের উত্তর দেওয়া। এটা দিয়ে শিখব — **একটা অভ্যন্তরীণ** (**internal**) **AI সহকারী** কীভাবে বানাতে হয়।

ShopBondhu — একটি ই-কমার্স প্রজেক্ট

বাংলাদেশি একটা অনলাইন শপিং প্ল্যাটফর্ম, যেখানে পণ্য, অর্ডার ও পেমেন্ট সব আছে। এখানেও আমরা পুরো প্ল্যাটফর্ম নয়, এর **AI সাপোর্ট চ্যাটবট** অংশটা নিয়ে কাজ করব — যা ক্রেতার প্রশ্নের উত্তর দেয় (অর্ডার কোথায়, পণ্যের তথ্য, রিটার্ন পলিসি)। এটা দিয়ে শিখব — **গ্রাহকমুখী** (**customer-facing**) **একটা চ্যাটবট** কীভাবে নির্ভরযোগ্য করে তুলতে হয়।

একটা জরুরি কথা — স্কোপ

লক্ষ্য করুন, আমরা পুরো ই-কমার্স বা SaaS অ্যাপ্লিকেশন বানাচ্ছি না। প্রতিটা ক্ষেত্রে আমরা একটা বড় বাস্তব প্রজেক্টের **একটা নির্দিষ্ট AI অংশ** (সহকারী বা চ্যাটবট) নিয়ে কাজ করছি — কারণ context engineering ঠিক সেখানেই ঘটে। এটাই বাস্তবে আপনি যেভাবে AI ফিচার যোগ করবেন।

কেন দুটো প্রজেক্ট?

SaaS আর ই-কমার্স — দুই জগতে context engineering-এর চাহিদা একটু আলাদা। দুটো দিয়ে শিখলে আপনি যেকোনো ধরনের AI অ্যাপে এই কৌশল প্রয়োগ করতে পারবেন।

Licensed to sakilorb@gmail.com · Do not redistribute

পর্ব ১

ভিত্তি গড়া

কন্টেক্সট ইঞ্জিনিয়ারিং শেখার আগে কয়েকটা মৌলিক ধারণা পরিষ্কার থাকা দরকার — কন্টেক্সট আসলে কী, একটা মডেল কীভাবে "ভাবে", আর তার স্মৃতির সীমা কতটুকু। এই পর্ব সেই শক্ত ভিত্তিটাই গড়ে দেবে।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০১

কন্টেক্সট আসলে কী?

মডেল যতই শক্তিশালী হোক, সঠিক কন্টেক্সট ছাড়া সে অন্ধের মতো। এই অধ্যায়ে আমরা একদম গোড়া থেকে বুঝব — কন্টেক্সট জিনিসটা কী, আর কেন এটাই একটা *AI* অ্যাপের সবচেয়ে গুরুত্বপূর্ণ অংশ।

১.১ একটা গল্প দিয়ে শুরু — নতুন কর্মীর প্রথম দিন

ধরুন ShopBondhu-তে আপনি একজন নতুন কাস্টমার সাপোর্ট কর্মী নিয়োগ দিলেন — নাম রিনা। রিনা ভীষণ বুদ্ধিমান, ভদ্র, আর দ্রুত শেখে। প্রথম দিনই আপনি তাকে বললেন, "একজন কাস্টমার রেগে আছেন, তার সমস্যাটা সামলাও।"

কিন্তু আপনি রিনাকে কিছুই দিলেন না — অর্ডার সিস্টেমে ঢোকার অনুমতি নেই, পণ্যের তালিকা সামনে নেই, রিটার্ন পলিসি কী তা সে জানে না, এমনকি কাস্টমার আগে কী কিনেছিল সেটাও তার অজানা। এখন রিনা যতই বুদ্ধিমান হোক, সে কী করবে? হয় চুপ করে থাকবে, নয়তো অনুমান করে এমন কিছু বলবে যা হয়তো ভুল।

এবার একই দৃশ্য আবার কল্পনা করুন। এবার রিনার ডেস্কে আপনি আগে থেকে সাজিয়ে রাখলেন — কাস্টমারের অর্ডার হিস্ট্রি, পণ্যের পূর্ণ তালিকা, কোম্পানির রিটার্ন পলিসি, আর একটা পরিষ্কার নির্দেশনা: "ভদ্রভাবে কথা বলো, শুধু আমাদের পলিসির ভেতরে থেকে সমাধান দাও।" এবার রিনা ঝলমলিয়ে উঠবে — সঠিক, দ্রুত, আর নির্ভরযোগ্য উত্তর দেবে।

রিনার বুদ্ধি দুই ক্ষেত্রেই সমান ছিল। বদলেছে শুধু একটা জিনিস — তার **কাজের পরিবেশ**, অর্থাৎ তাকে দেওয়া তথ্য আর নির্দেশনা।

> **মূল কথা**
>
> একটা ভাষা মডেল হুবহু এই রিনার মতো — অসম্ভব মেধাবী, কিন্তু সঠিক কন্টেক্সট ছাড়া অন্ধ। Context Engineering মানে হলো মডেল কাজ শুরু করার আগেই তার "ডেস্ক" ঠিকঠাক সাজিয়ে দেওয়া।

Licensed to sakilorb@gmail.com · Do not redistribute

১.২ LLM কীভাবে "ভাবে" — টোকেন ও context window

রিনার সাথে মডেলের একটা বড় পার্থক্য আছে, আর সেটা না বুঝলে context engineering শেখা কঠিন। পার্থক্যটা হলো — **মডেলের কোনো স্মৃতি নেই**। রিনা গতকালের কথোপকথন মনে রাখতে পারে, কিন্তু একটা ভাষা মডেল প্রতিবার যেন নতুন করে জন্ম নেয়।

> **উপমা**
>
> মডেলকে ভাবুন একজন ভয়াবহ স্মৃতিভ্রংশ (amnesia) রোগী হিসেবে — কিন্তু অসম্ভব জ্ঞানী। প্রতিবার তাকে প্রশ্ন করলে সে আগের সব কথা ভুলে যায়। সে শুধু ততটুকুই জানে, যতটুকু আপনি ঠিক ওই মুহূর্তে তার সামনে রাখেন।

আপনি যা কিছু তার সামনে রাখেন — প্রশ্ন, নির্দেশনা, তথ্য, আগের কথোপকথন — সব মিলিয়ে যে জায়গাটায় সেগুলো ধরে, তাকে বলে **context window**। এটা মডেলের "ডেস্ক"। এই ডেস্কে যা থাকে, মডেল শুধু তাটুকুই দেখতে পায়; এর বাইরের কিছু তার কাছে অস্তিত্বহীন।

কিন্তু এই ডেস্কের একটা মাপ আছে। মডেল লেখাকে গোটা শব্দ হিসেবে নয়, ছোট ছোট টুকরো হিসেবে পড়ে — এই টুকরোগুলোকে বলে **টোকেন** (**token**)। মোটামুটি একটা ইংরেজি শব্দ গড়ে ১ থেকে ১.৫ টোকেন; বাংলা লেখা সাধারণত আরও বেশি টোকেন খরচ করে। context window-এর মাপও টোকেনেই হিসাব হয় — যেমন "এই মডেল ১,২৮,০০০ টোকেন পর্যন্ত নিতে পারে।"

নিচের কোডে দেখুন — মডেলকে পাঠানো `messages` তালিকাটাই আসলে তার পুরো কন্টেক্সট। এর বাইরে সে কিছুই জানে না:

```
# মডেলকে যা পাঠাই, সেটাই তার সম্পূর্ণ "জগৎ"
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=500,
    messages=[
        {"role": "user", "content": "আমার অর্ডার কোথায়?"}
    ]
)
# এখানে মডেল জানে না কে এই ইউজার, তার অর্ডার কী —
# কারণ সেই তথ্য আমরা context window-এ দিইনি।
```

কোড ১.১ — শুধু প্রশ্ন পাঠালে মডেলের কাছে কোনো প্রাসঙ্গিক তথ্য থাকে না।

> **সাবধান**

Licensed to sakilorb@gmail.com · Do not redistribute

যেহেতু মডেলের নিজস্ব স্মৃতি নেই, প্রতিটা request-এ প্রয়োজনীয় সব তথ্য আবার করে পাঠাতে হয়। "গতবার তো বলেছিলাম" — এই যুক্তি মডেলের কাছে অর্থহীন।

১.৩ কন্টেক্সট = মডেলের কাজের পরিবেশ

এখন একটা ভুল ধারণা ভেঙে দিই। অনেকে মনে করেন "কন্টেক্সট মানে তো শুধু প্রশ্নটা"। আসলে প্রশ্ন কন্টেক্সটের একটা ছোট অংশমাত্র। সম্পূর্ণ কন্টেক্সট হলো মডেলের পুরো কাজের পরিবেশ, যেখানে সাধারণত এই উপাদানগুলো থাকে:

একটি কন্টেক্সটের ৬টি সাধারণ উপাদান

১. **System নির্দেশনা** — মডেলের ভূমিকা ও নিয়ম। যেমন: "তুমি ShopBondhu-র ভদ্র সাপোর্ট সহকারী।"

২. **প্রাসঙ্গিক তথ্য** (**data**) — ডেটাবেস বা ডকুমেন্ট থেকে আনা তথ্য। যেমন কাস্টমারের অর্ডার রেকর্ড।

৩. **উদাহরণ** (**examples**) — কয়েকটা নমুনা ইনপুট-আউটপুট, যাতে মডেল ধরন বোঝে।

৪. **কথোপকথনের ইতিহাস** — আগের মেসেজগুলো, যাতে আলোচনার ধারাবাহিকতা থাকে।

৫. **টুল** (**tools**) — মডেল কী কী কাজ করতে পারবে, যেমন অর্ডার স্ট্যাটাস খোঁজা।

৬. **আউটপুট ফরম্যাট** — উত্তর কেমন কাঠামোয় চাই, যেমন JSON।

Context Engineering হলো এই উপাদানগুলোকে বুদ্ধিমত্তার সাথে বাছাই করা, সাজানো আর ভারসাম্য রাখার শিল্প। কোন তথ্য রাখব, কোনটা বাদ দেব, কোনটা আগে রাখব — এই সিদ্ধান্তগুলোই আপনার অ্যাপের মান ঠিক করে দেয়।

আমাদের দুই প্রজেক্টের দিকে তাকান। TaskFlow-এর AI সহকারীর জন্য হয়তো বেশি দরকার ব্যবহারকারীর প্রজেক্ট ডেটা আর কাজের ইতিহাস। অন্যদিকে ShopBondhu-র চ্যাটবটের জন্য বেশি দরকার পণ্যের তথ্য, অর্ডার স্ট্যাটাস আর পলিসি। উপাদানগুলো একই, কিন্তু কোনটায় কতটা জোর দেব — সেটা প্রজেক্টভেদে বদলায়। এই সূক্ষ্ম সিদ্ধান্তগুলোই আমরা পরের অধ্যায়গুলোতে একটা একটা করে শিখব।

মনে রাখুন

প্রম্পট লেখা context engineering-এর একটা অংশ মাত্র। পুরো ছবিটা হলো — মডেলের চারপাশের গোটা পরিবেশটাকে সচেতনভাবে ডিজাইন করা।

Licensed to sakilorb@gmail.com · Do not redistribute

কী শিখলেন

- ✓ মডেল মেধাবী হলেও সঠিক কন্টেক্সট ছাড়া কার্যকর নয় — ঠিক নতুন কর্মী রিনার মতো।
- ✓ মডেলের নিজস্ব স্মৃতি নেই; সে শুধু context window-এ থাকা তথ্যটুকুই দেখে।
- ✓ লেখা টোকেনে ভাঙা হয়, আর context window-এর একটা নির্দিষ্ট মাপ আছে।
- ✓ কন্টেক্সট মানে শুধু প্রশ্ন নয় — এতে থাকে নির্দেশনা, তথ্য, উদাহরণ, ইতিহাস, টুল ও ফরম্যাট।
- ✓ Context Engineering হলো এই উপাদানগুলো বাছাই ও সাজানোর শিল্প।

পরের অধ্যায়ে আমরা দেখব — *Prompt Engineering* আর *Context Engineering* আসলে কীভাবে আলাদা, আর কখন কোনটা আপনার বেশি কাজে লাগবে।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০২

Prompt Engineering বনাম Context Engineering

অনেকে এই দুটো শব্দ গুলিয়ে ফেলেন, কেউ কেউ ভাবেন এরা একই জিনিস। আসলে এরা সম্পর্কিত, কিন্তু এক নয়। এই অধ্যায়ে আমরা পার্থক্যটা পরিষ্কার করব — আর বুঝব কখন কোনটা আপনার বেশি কাজে লাগবে।

২.১ দুটোর আসল পার্থক্য

সহজ কথায়, **Prompt Engineering** হলো মডেলকে দেওয়া নির্দেশনা বা প্রশ্নটা ঠিকঠাক ভাষায় লেখার কৌশল। এখানে মনোযোগ থাকে একটা মেসেজের ওপর — শব্দচয়ন, স্পষ্টতা, কীভাবে চাইলে মডেল ভালো উত্তর দেয়।

অন্যদিকে **Context Engineering** অনেক বড় পরিসরের ব্যাপার। এখানে আপনি শুধু প্রশ্ন নয়, মডেলের চারপাশের পুরো পরিবেশটা ডিজাইন করেন — কোন তথ্য দেবেন, কোন উদাহরণ, কোন ইতিহাস, কোন টুল, কোন ফরম্যাট, আর তার সাথে প্রম্পট। অর্থাৎ প্রম্পট ইঞ্জিনিয়ারিং আসলে কন্টেক্সট ইঞ্জিনিয়ারিংয়েরই একটা অংশ।

> **উপমা**
>
> একজন বিশেষজ্ঞ পরামর্শককে ভালো করে একটা প্রশ্ন করা — এটা prompt engineering। আর তিনি বসার আগেই তাঁর অফিস সাজিয়ে, দরকারি ফাইল টেবিলে রেখে, প্রয়োজনীয় টুল হাতের কাছে দিয়ে পুরো ব্রিফিং তৈরি করে রাখা — এটা context engineering।

নিচের তুলনাটা দেখলে পার্থক্যটা একদম স্পষ্ট হয়ে যাবে:

সংকীর্ণ পরিসর

Prompt Engineering

- ফোকাস: একটা প্রশ্ন বা নির্দেশনা
- মূল কাজ: শব্দচয়ন ও স্পষ্টতা
- সাধারণত single-turn
- দ্রুত, হালকা কাজে যথেষ্ট

বিস্তৃত পরিসর

Context Engineering

- ফোকাস: পুরো পরিবেশ
- মূল কাজ: তথ্য, উদাহরণ, টুল সাজানো
- প্রায়ই multi-turn ও system-level
- বাস্তব অ্যাপ বানাতে অপরিহার্য

Licensed to sakilorb@gmail.com · Do not redistribute

মনে রাখুন

ভালো প্রম্পট লেখা জরুরি — কিন্তু সেটা পুরো গল্পের একটা পাতা মাত্র। পুরো বইটা হলো কন্টেক্সট ইঞ্জিনিয়ারিং।

২.২ কখন কোনটা দরকার

সব কাজে ভারী আয়োজন লাগে না। ছোট, একবারের কাজে শুধু ভালো প্রম্পটই যথেষ্ট। যেমন — "এই অনুচ্ছেদটা তিন লাইনে সংক্ষেপ করো" বা "এই ইমেইলটা ভদ্রভাবে আবার লেখো।" এখানে কোনো বাইরের ডেটা, ইতিহাস বা টুল লাগছে না।

কিন্তু যখন আপনি একটা **বাস্তব অ্যাপ** বানাবেন — যেখানে লাইভ ডেটা আসে, একাধিক ধাপে কথোপকথন চলে, নানা রকম পরিস্থিতি সামলাতে হয় — তখন শুধু প্রম্পট দিয়ে কাজ চলবে না। তখনই context engineering দরকার।

একটা সহজ নিয়ম মাথায় রাখুন: যত বেশি বাস্তব আর জটিল হবে আপনার অ্যাপ, তত বেশি প্রম্পট থেকে কন্টেক্সটের দিকে সরে যেতে হবে।

আমাদের প্রজেক্টে এর প্রয়োগ

P **শুধু প্রম্পট যথেষ্ট** TaskFlow-এ একজন ব্যবহারকারী একটা প্রজেক্টের বর্ণনা পেস্ট করে বলল "এক লাইনে সারাংশ দাও।" কোনো বাইরের তথ্য লাগছে না।

C **কন্টেক্সট দরকার**: ShopBondhu-র সাপোর্ট বট একজন ক্রেতার আসল অর্ডার নিয়ে প্রশ্নের উত্তর দিচ্ছে — এখানে লাইভ ডেটা, পলিসি ও ইতিহাস সব সাজিয়ে দিতে হবে।

২.৩ হাতে-কলমে: ভুল প্রম্পট vs সাজানো কন্টেক্সট

তত্ত্ব যথেষ্ট হলো, এবার কোড দিয়ে দেখি। ধরা যাক ShopBondhu-তে একজন ক্রেতা জিজ্ঞেস করল, "আমার অর্ডারটা কবে আসবে?" অনেক নতুন ডেভেলপার ভাবেন — প্রম্পটটা আরও সুন্দর করে লিখলেই সমস্যা মিটে যাবে। দেখা যাক সেটা কাজ করে কিনা।

চেষ্টা ১ — শুধু প্রম্পট আরও "ভালো" করে লেখা:

```
# প্রম্পট সুন্দর, কিন্তু কোনো অর্ডার ডেটা নেই
messages = [
    {"role": "system",
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
        "content": "তুমি একজন দক্ষ, ভদ্র সাপোর্ট এজেন্ট। সুন্দর করে উত্তর দাও।"},
    {"role": "user", "content": "আমার অর্ডারটা কবে আসবে?"}
]
# ফলাফল: মডেল ভদ্রভাবে কথা বলবে, কিন্তু আসল তারিখ
# বলতে পারবে না — কারণ সে অর্ডারটাই দেখছে না।
```

কোড ২.১ — প্রম্পট যত সুন্দরই হোক, ডেটা ছাড়া সঠিক উত্তর অসম্ভব।

> **সাবধান**
>
> ডেটা না থাকলে মডেল প্রায়ই অনুমান করে একটা তারিখ বানিয়ে বলে দেয় (hallucination)। ক্রেতার কাছে এটা মিথ্যা তথ্য — যা আপনার ব্যবসার বিশ্বাস নষ্ট করে।

চেষ্টা ২ — কন্টেক্সট সাজিয়ে দেওয়া: এবার আমরা ডেটাবেস থেকে ক্রেতার আসল অর্ডার তথ্য টেনে এনে কন্টেক্সটে যোগ করব, আর পরিষ্কার নিয়ম দেব।

```
# ডেটাবেস থেকে আসল অর্ডার টেনে আনলাম
order = db.get_order(user_id)   # {id, পণ্য, স্ট্যাটাস, ডেলিভারি তারিখ}

messages = [
    {"role": "system", "content": (
        "তুমি ShopBondhu-র সাপোর্ট এজেন্ট। শুধু নিচের অর্ডার "
        "তথ্যের ভিত্তিতে উত্তর দাও। তথ্যে না থাকলে অনুমান করো না।"
    )},
    {"role": "system", "content": format_order(order)},  # আসল ডেটা
    {"role": "user", "content": "আমার অর্ডারটা কবে আসবে?"}
]
# ফলাফল: "আপনার #SB-2043 অর্ডারটি ১৮ জুন ডেলিভারি হবে।"
# — সঠিক, নির্ভরযোগ্য, এবং অনুমানমুক্ত।
```

কোড ২.২ — সঠিক তথ্য কন্টেক্সটে দিলে উত্তরও সঠিক হয়।

লক্ষ্য করুন, দুই ক্ষেত্রে মডেল একই। শুধু দ্বিতীয়বার আমরা তাকে আসল অর্ডার ডেটা আর একটা পরিষ্কার নিয়ম দিয়েছি। এই ছোট পার্থক্যটাই একটা অকেজো বটকে একটা নির্ভরযোগ্য সহকারীতে বদলে দেয়।

> **মূল শিক্ষা**
>
> যত চতুর শব্দই বসান, সঠিক তথ্যের অভাব পূরণ হয় না। প্রম্পটের ভাষা গুরুত্বপূর্ণ, কিন্তু কন্টেক্সটের পূর্ণতা তার চেয়েও বেশি গুরুত্বপূর্ণ।

Licensed to sakilorb@gmail.com · Do not redistribute

কী শিখলেন

- ✓ Prompt engineering = একটা নির্দেশনার ভাষা; Context engineering = পুরো পরিবেশের ডিজাইন।
- ✓ প্রম্পট ইঞ্জিনিয়ারিং আসলে কন্টেক্সট ইঞ্জিনিয়ারিংয়ের একটা অংশ।
- ✓ ছোট, একবারের কাজে ভালো প্রম্পটই যথেষ্ট বাস্তব অ্যাপে কন্টেক্সট লাগে।
- ✓ যত জটিল অ্যাপ, তত বেশি প্রম্পট থেকে কন্টেক্সটের দিকে সরতে হয়।
- ✓ সুন্দর শব্দ সঠিক তথ্যের বিকল্প নয় — কন্টেক্সটের পূর্ণতাই আসল।

পরের অধ্যায়ে আমরা *context window*-এর ভেতরে ঢুকব — টোকেন কীভাবে গোনা হয়, কেন সব তথ্য একসাথে দেওয়া যায় না, আর *"context rot"* নামের একটা লুকানো বিপদ।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৩

Context Window — মডেলের স্মৃতির সীমা

মডেলের ডেস্ক যত বড়ই হোক, তার একটা সীমা আছে। এই অধ্যায়ে আমরা বুঝব টোকেন কীভাবে গোনা হয়, কেন সব তথ্য একসাথে দেওয়া যায় না, *"context rot"* নামের লুকানো বিপদটা কী, আর কন্টেক্সট কীভাবে সরাসরি আপনার খরচ ও গতির ওপর প্রভাব ফেলে।

৩.১ টোকেন কী, কীভাবে গোনা হয়

প্রথম অধ্যায়ে আমরা জেনেছি, মডেল লেখাকে ছোট ছোট টুকরো বা **টোকেনে** ভেঙে পড়ে। এবার একটু গভীরে যাই। সাধারণ ও পরিচিত শব্দ প্রায়ই একটা টোকেন হয়, কিন্তু লম্বা বা অপরিচিত শব্দ কয়েক টুকরোয় ভাগ হয়। স্পেস আর যতিচিহ্নও টোকেন খরচ করে।

মোটামুটি হিসাব হিসেবে ইংরেজিতে প্রায় **৪টি অক্ষর = ১ টোকেন**। কিন্তু এখানে বাংলা ব্যবহারকারীদের জন্য একটা জরুরি কথা আছে — বাংলা লেখা সাধারণত ইংরেজির চেয়ে অনেক বেশি টোকেন খরচ করে। কারণ বেশিরভাগ টোকেনাইজার বাংলা যুক্তাক্ষর ও কারগুলোকে একাধিক টুকরোয় ভাঙে। ফলে একই অর্থের একটা বাংলা অনুচ্ছেদ ইংরেজির তুলনায় দুই-তিন গুণ টোকেন নিতে পারে।

> **মনে রাখুন**
>
> ইনপুট আর আউটপুট — দুটোই টোকেন হিসেবে গোনা হয়। আপনি যা পাঠান এবং মডেল যা ফেরত দেয়, সব মিলিয়েই হিসাব।

প্রতিটা মডেলের একটা সর্বোচ্চ context window থাকে — যেমন "এই মডেল ১,২৮,০০০ টোকেন পর্যন্ত নিতে পারে।" কোড লেখার আগে আনুমানিক টোকেন গুনে নেওয়া ভালো অভ্যাস:

```
# আনুমানিক টোকেন গোনার সহজ উপায়
text = open("shopbondhu_policy.txt").read()

# মোটামুটি আন্দাজ: প্রতি ৪ অক্ষরে ১ টোকেন (ইংরেজি)
approx_tokens = len(text) // 4
print(f"আনুমানিক টোকেন: {approx_tokens}")
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
# নির্ভুল গণনার জন্য মডেল-প্রোভাইডারের
# token-counting টুল ব্যবহার করুন।
```

কোড ৩.১ — কন্টেক্সটে কিছু রাখার আগে তার "ওজন" আন্দাজ করে নিন।

৩.২ কেন সব তথ্য একসাথে দেওয়া যায় না

স্বাভাবিক একটা প্রশ্ন আসে — "context window যদি অনেক বড় হয়, তাহলে সব তথ্য একসাথে ঢেলে দিলেই তো হয়?" এর দুটো বড় সমস্যা আছে।

প্রথমত, একটা শক্ত সীমা আছে। context window-এর মাপ পেরিয়ে গেলে request হয় ব্যর্থ হবে, নয়তো তথ্যের কিছু অংশ কেটে বাদ পড়বে (truncate) — আর তখন গুরুত্বপূর্ণ কিছু হারিয়ে যেতে পারে।

দ্বিতীয়ত, বেশি মানেই ভালো নয়। সীমার ভেতরে থাকলেও, অপ্রয়োজনীয় তথ্যের স্তূপ মডেলকে বিভ্রান্ত করে। তাকে তখন গুরুত্বপূর্ণ তথ্য খুঁজে বের করতে হয় অজস্র অপ্রাসঙ্গিক তথ্যের ভিড় থেকে।

> **উপমা**
>
> একটা পরিচ্ছন্ন ডেস্কে শুধু দরকারি ৩টা ফাইল রাখা — আর একটা ডেস্কে অফিসের সব ফাইল স্তূপ করে রাখা। দ্বিতীয় ডেস্কে দরকারি ফাইলটা খুঁজে পাওয়াই কঠিন। মডেলের জন্যও ঠিক তা-ই।

গবেষণায় দেখা গেছে, মডেল কন্টেক্সটের শুরু আর শেষের তথ্যে বেশি মনোযোগ দেয়; বিশাল কন্টেক্সটের **মাঝখানে** চাপা পড়া তথ্য প্রায়ই উপেক্ষিত হয়। একে বলে "lost in the middle"। তাই গুরুত্বপূর্ণ তথ্য কন্টেক্সটের কোথায় রাখছেন, সেটাও ভাবার বিষয়।

৩.৩ "Context Rot" — বেশি তথ্যের বিপদ

এখানে একটা গুরুত্বপূর্ণ ধারণা আসে — **Context Rot**। যত বেশি অপ্রাসঙ্গিক, পুরোনো বা পরস্পরবিরোধী তথ্য আপনি কন্টেক্সটে ঢালেন, মডেলের উত্তরের মান তত খারাপ হতে থাকে। সে বিভ্রান্ত হয়, তথ্য গুলিয়ে ফেলে, এমনকি নিজের কথার সাথে নিজেই বিরোধ করে।

> **বাস্তব উদাহরণ**
>
> ShopBondhu-র বটকে প্রতিটা প্রশ্নের জন্য পুরো ৫,০০০ পণ্যের ক্যাটালগ ঢেলে দিলে সে ধীর হবে, খরচ বাড়বে, আর প্রায়ই ভুল পণ্য ধরে উত্তর দেবে। ভালো উপায় — শুধু প্রাসঙ্গিক কয়েকটা পণ্য টেনে আনা (এটাই RAG, আমরা অধ্যায় ৭-এ শিখব)।

Licensed to sakilorb@gmail.com · Do not redistribute

একইভাবে TaskFlow-এ ব্যবহারকারীর দুই বছরের পুরো টাস্ক ইতিহাস না দিয়ে, শুধু চলতি প্রজেক্টের সাম্প্রতিক কাজগুলো দিলে মডেল অনেক নিখুঁত উত্তর দেবে। মূল নিয়মটা সহজ:

সোনালি নিয়ম

প্রাসঙ্গিকতা > পরিমাণ। বেশি তথ্য দেওয়া নয়, বরং **সঠিক** তথ্য দেওয়াই লক্ষ্য।

৩.৪ খরচ ও গতির হিসাব

টোকেন শুধু একটা কারিগরি ধারণা নয় — টোকেন মানে সরাসরি **টাকা আর সময়**। প্রায় সব API ইনপুট ও আউটপুট টোকেন অনুযায়ী চার্জ করে। কন্টেক্সট যত বড় প্রতিটা কলের খরচ তত বেশি। আবার বেশি টোকেন মানে মডেলের উত্তর দিতেও বেশি সময় (latency)।

ছোট স্কেলে এটা হয়তো গায়ে লাগে না, কিন্তু হাজার হাজার ব্যবহারকারী হলে একটা ফোলানো কন্টেক্সট খরচকে কয়েকগুণ বাড়িয়ে দেয়। নিচের তুলনাটা দেখুন — একই প্রশ্নের জন্য কন্টেক্সট কতটা সাজানো, তার ওপর প্রতি কলের আপেক্ষিক খরচ কেমন বদলায়:



লক্ষ্য করুন, কন্টেক্সট ছেঁটে ফেলা এখানে একটা বিরল win-win — খরচ কমে, গতি বাড়ে আর আগের অংশে দেখা context rot-ও কমে বলে উত্তরের মানও ভালো হয়। ভালো context engineering তাই শুধু কারিগরি দক্ষতা নয়, এটা সরাসরি ব্যবসায়িক বুদ্ধিমত্তাও।

কী শিখলেন

- ✓ লেখা টোকেনে ভাঙা হয়; বাংলা ইংরেজির চেয়ে বেশি টোকেন খরচ করে।
- ✓ ইনপুট ও আউটপুট — দুটোই টোকেন হিসেবে গোনা হয়।
- ✓ context window-এর শক্ত সীমা আছে, আর বেশি তথ্য মানেই ভালো নয়।
- ✓ অপ্রাসঙ্গিক/পুরোনো তথ্যের স্তূপ "context rot" ঘটায় — মান কমে।

Licensed to sakilorb@gmail.com · Do not redistribute

✓ কন্টেক্সট ছেঁটে রাখলে খরচ কমে, গতি বাড়ে আর উত্তরও ভালো হয়।

এই অধ্যায়ের সাথে শেষ হলো আমাদের ভিত্তি-পর্ব। পরের পর্বে আমরা কন্টেক্সটের উপাদানগুলো একটা একটা করে গড়তে শিখব — শুরু হবে *System Prompt* ও *Instruction* ডিজাইন দিয়ে।

Licensed to sakilorb@gmail.com · Do not redistribute

পর্ব ২

কন্টেক্সটের উপাদান

ভিত্তি তো গড়া হলো। এবার আমরা কন্টেক্সটের প্রধান উপাদানগুলো একটা একটা করে হাতে গড়তে শিখব — মডেলের ভূমিকা ঠিক করা (system prompt), উদাহরণ দিয়ে শেখানো (few-shot), আর উত্তরের কাঠামো নিয়ন্ত্রণ (output format)। এগুলোই একটা অ্যাপের মূল কাঠামো।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৪

System Prompt ও Instruction ডিজাইন

System prompt হলো মডেলের "চাকরির বিবরণ" — তার ভূমিকা, ব্যক্তিত্ব, নিয়ম আর সীমা। একটা ভালো *system prompt* একটা এলোমেলো মডেলকে নির্ভরযোগ্য, নির্দিষ্ট আচরণের সহকারীতে বদলে দেয়। এই অধ্যায়ে আমরা সেটা ধাপে ধাপে গড়তে শিখব।

৪.১ ভালো system prompt-এর শর্ত

System prompt হলো সবার উপরের নির্দেশনা, যা পুরো কথোপকথনজুড়ে প্রযোজ্য থাকে। এটা সাধারণ user মেসেজ থেকে আলাদা — user মেসেজ বদলায়, কিন্তু system prompt স্থায়ী ব্রিফের মতো ভূমিকা পালন করে। মডেল প্রতিটা উত্তর দেওয়ার সময় এই ব্রিফটা মাথায় রাখে।

একটা ভালো system prompt-এর কয়েকটা গুণ থাকে:

ভালো system prompt-এর ৫টি শর্ত

১. **পরিষ্কার ভূমিকা** — মডেল কে, কার হয়ে কথা বলছে।

২. **সুনির্দিষ্ট নিয়ম** — অস্পষ্ট নয়, মাপা ও স্পষ্ট নির্দেশনা।

৩. **সীমা নির্ধারণ** — কী করবে না, কখন মানুষের কাছে পাঠাবে।

৪. **টোন ও ভাষা** — কেমন সুরে, কোন ভাষায় কথা বলবে।

৫. **সংক্ষিপ্ত কিন্তু সম্পূর্ণ** — উপন্যাস নয়, তবে দরকারি সব কভার করা।

সবচেয়ে সাধারণ ভুলটা হলো অস্পষ্টতা। "সাহায্যকারী হও" — এই নির্দেশনা মডেলকে কিছুই শেখায় না। তার বদলে মডেলকে ঠিক কী করতে হবে, সেটা পরিষ্কার করে বলতে হয়।

৪.২ ভূমিকা, নিয়ম ও সীমা ঠিক করা

একটা শক্তিশালী system prompt-কে আমরা কয়েকটা ব্লকে ভেঙে ভাবতে পারি — পরিচয়, লক্ষ্য, আচরণের নিয়ম, সীমা, আর টোন। প্রতিটা ব্লক আলাদা করে ভাবলে কিছু বাদ পড়ে না।

Licensed to sakilorb@gmail.com · Do not redistribute

লেখার সময় দুটো অভ্যাস খুব কাজে দেয়। **প্রথমত, ইতিবাচকভাবে বলুন।** "অভদ্রতা করবে না"-এর বদলে লিখুন "সবসময় ভদ্র ও সহানুভূতিশীল থাকো" — মডেল ইতিবাচক নির্দেশনা ভালো অনুসরণ করে।

দুর্বল — অস্পষ্ট

যা এড়াবেন

- "সাহায্যকারী ও ভালো হও"
- "সংক্ষেপে উত্তর দাও"
- "অভদ্র হবে না"
- "সবকিছুর উত্তর দাও"

শক্তিশালী — সুনির্দিষ্ট

যা করবেন

- "তুমি ShopBondhu-র সাপোর্ট এজেন্ট"
- "৩-৪ বাক্যে উত্তর দাও"
- "সবসময় ভদ্র ও সহানুভূতিশীল থাকো"
- "পলিসির বাইরে হলে মানুষের কাছে পাঠাও"

দ্বিতীয়ত, সুনির্দিষ্ট হোন। "সংক্ষেপে উত্তর দাও" অস্পষ্ট"৩-৪ বাক্যে উত্তর দাও" মাপা। গ্রাহকমুখী বটের জন্য সীমা নির্ধারণ বিশেষভাবে জরুরি — যেমন পলিসির বাইরে কোনো প্রতিশ্রুতি না দেওয়া, বা রেগে যাওয়া ক্রেতাকে মানুষের কাছে হস্তান্তর করা।

৪.৩ ShopBondhu সাপোর্ট বটের system prompt তৈরি

এবার সব ব্লক মিলিয়ে আমরা ShopBondhu-র জন্য একটা পূর্ণ system prompt গড়ি। প্রথমে দেখুন একজন নতুন ডেভেলপার যা লেখেন — একটা দুর্বল এক-লাইনের ব্রিফ:

```
# দুর্বল system prompt
SYSTEM = "তুমি একটা সহায়ক বট। কাস্টমারকে সাহায্য করো।"
```

কোড ৪.১ — এত অস্পষ্ট যে মডেল নিজের মতো করে আচরণ করবে।

এবার একই বটের জন্য একটা সাজানো, ব্লক-ভিত্তিক system prompt দেখুন:

```
● SHOPBONDHU — SYSTEM PROMPT

# পরিচয়
তুমি ShopBondhu-র কাস্টমার সাপোর্ট এজেন্ট, নাম "বন্ধু"।

# লক্ষ্য
ক্রেতার প্রশ্নের সঠিক ও দ্রুত সমাধান দেওয়া।

# নিয়ম
• সবসময় ভদ্র, সহানুভূতিশীল ও বাংলায় কথা বলো।
• শুধু দেওয়া অর্ডার ও পলিসি তথ্যের ভিত্তিতে উত্তর দাও।
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
• তথ্যে না থাকলে অনুমান করো না — সততার সাথে জানাও।
• উত্তর ৩-৪ বাক্যে রাখো।

# সীমা
• পলিসির বাইরে কোনো রিফান্ড/ছাড়ের প্রতিশ্রুতি দিও না।
• ক্রেতা খুব রেগে গেলে বা জটিল সমস্যা হলে মানুষ-এজেন্টের কাছে পাঠাও।
```

এই prompt-টা messages তালিকায় কীভাবে বসে, দেখুন:

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},    # উপরের ব্রিফ
    {"role": "system", "content": format_order(order)}, # আসল ডেটা
    {"role": "user", "content": user_message}
]
```

কোড ৪.২ — সাজানো system prompt + ডেটা + ব্যবহারকারীর প্রশ্ন।

> **মনে রাখুন**
>
> প্রথমবারেই নিখুঁত system prompt লেখা যায় না। এটা একটা পুনরাবৃত্তির কাজ — বাস্তব প্রশ্ন দিয়ে পরীক্ষা করুন, যেখানে মডেল ভুল করে সেখানে নিয়ম যোগ বা স্পষ্ট করুন।

TaskFlow-এও একই নীতি। এর অভ্যন্তরীণ AI সহকারীর জন্যও একটা সাজানো system prompt লাগে — শুধু ভূমিকা ও টোন আলাদা, কারণ এটা গ্রাহক নয়, টিম-সদস্যদের সাহায্য করে:

```
● TASKFLOW – SYSTEM PROMPT

# পরিচয়
তুমি TaskFlow-র অভ্যন্তরীণ প্রজেক্ট সহকারী।

# নিয়ম
• শুধু দেওয়া প্রজেক্ট ও টাস্ক ডেটার ভিত্তিতে উত্তর দাও।
• সংক্ষিপ্ত, কাজের উপযোগী ভাষায় বলো — অপ্রয়োজনীয় ভণিতা নয়।
• অগ্রাধিকার বা ডেডলাইন নিয়ে নিশ্চিত না হলে অনুমান করো না।
```

লক্ষ্য করুন, ShopBondhu-র বট ভদ্র ও সহানুভূতিশীল, কিন্তু TaskFlow-র সহকারী সংক্ষিপ্ত ও কাজমুখী। একই কাঠামো, ভিন্ন প্রজেক্ট — সেই অনুযায়ী টোন বদলায়।

Licensed to sakilorb@gmail.com · Do not redistribute

কী শিখলেন

- ✓ System prompt হলো পুরো কথোপকথনজুড়ে প্রযোজ্য স্থায়ী ব্রিফ।
- ✓ ভালো prompt-এ থাকে পরিষ্কার ভূমিকা, সুনির্দিষ্ট নিয়ম, সীমা ও টোন।
- ✓ ইতিবাচক ও সুনির্দিষ্ট ভাষা অস্পষ্ট নির্দেশনার চেয়ে ভালো কাজ করে।
- ✓ গ্রাহকমুখী বটে সীমা নির্ধারণ (কখন থামবে/হস্তান্তর করবে) অপরিহার্য।
- ✓ System prompt পরীক্ষা করে করে ধীরে ধীরে উন্নত করতে হয়।

পরের অধ্যায়ে আমরা শিখব *few-shot examples* — কীভাবে কয়েকটা নমুনা দেখিয়ে মডেলকে ঠিক আমাদের চাওয়া ধরনে কাজ করাতে হয়।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৫

Few-shot Examples — উদাহরণ দিয়ে শেখানো

কখনো কখনো মডেলকে কী করতে হবে তা শুধু বলে দেওয়া যথেষ্ট নয় — কয়েকটা নমুনা দেখিয়ে দিলে কাজটা অনেক ভালো হয়। এই অধ্যায়ে আমরা শিখব *few-shot examples* কীভাবে মডেলকে ঠিক আমাদের চাওয়া ধরনে কাজ করায়।

৫.১ Zero-shot বনাম few-shot

মডেলকে কোনো উদাহরণ ছাড়া সরাসরি কাজ দেওয়াকে বলে **zero-shot**। এখানে মডেল তার সাধারণ প্রশিক্ষণের ওপর ভর করে উত্তর দেয়। পরিচিত, সাধারণ কাজে এটা ভালোই কাজ করে।

আর কাজটা করার কয়েকটা নমুনা (উদাহরণ) আগে দেখিয়ে, তারপর আসল কাজ দেওয়াকে বলে **few-shot**। মডেল নমুনাগুলোর ধরন দেখে সেই অনুযায়ী উত্তর দেয়।

> **উপমা**
>
> কাউকে নতুন একটা কাজ শেখানোর দুটো উপায় — মুখে বুঝিয়ে দেওয়া (zero-shot), অথবা ২-৩টা করা উদাহরণ দেখিয়ে তারপর তাকে করতে বলা (few-shot)। দেখিয়ে দেওয়াটা প্রায়ই অনেক বেশি পরিষ্কার।

Few-shot বিশেষভাবে কাজে লাগে যখন আপনি একটা নির্দিষ্ট ফরম্যাট চান, একটা বিশেষ টোন চান, কিংবা ডোমেইন-নির্দিষ্ট কোনো শ্রেণীবিন্যাস ধারাবাহিকভাবে চান। নমুনা দেখলে মডেল শুধু কাজটা নয়, কাজের ধরনটাও নকল করে।

৫.২ ভালো উদাহরণ বাছাইয়ের নিয়ম

Few-shot-এ উদাহরণের সংখ্যা নয়, মান-ই আসল। সাধারণত ২ থেকে ৫টা ভালো উদাহরণই যথেষ্ট। বাছাই করার সময় কয়েকটা নিয়ম মাথায় রাখুন:

ভালো উদাহরণের ৫টি নিয়ম

১ **প্রতিনিধিত্বমূলক হোক** — বাস্তবে যেসব পরিস্থিতি আসবে, উদাহরণে তার বৈচিত্র্য থাকুক।

Licensed to sakilorb@gmail.com · Do not redistribute

২ **ফরম্যাট অভিন্ন হোক** — আপনি যে কাঠামোয় উত্তর চান, উদাহরণও ঠিক সেই কাঠামোয় হোক।

৩ **ভারসাম্য রাখুন** — সব উদাহরণ এক শ্রেণির হলে মডেল ওই শ্রেণিতেই ঝুঁকে পড়বে।

৪ **নির্ভুল হোক** — ভুল উদাহরণ মডেলকে ভুল আচরণই শেখায়।

৫ **টোকেন খেয়াল রাখুন** — উদাহরণ টোকেন খরচ করে; সমৃদ্ধি ও খরচের ভারসাম্য রাখুন।

> **সাবধান**
>
> যদি আপনার ৪টা উদাহরণের সবগুলোই "বাগ" শ্রেণির হয়, মডেল প্রায় সব কিছুকেই "বাগ" বলে ফেলবে। উদাহরণে শ্রেণিগুলোর ভারসাম্য রাখা তাই অত্যন্ত জরুরি।

৫.৩ TaskFlow: ইমেইল শ্রেণীবিন্যাস উদাহরণ

এবার বাস্তব উদাহরণ। TaskFlow-এ ব্যবহারকারীদের পাঠানো ইমেইল স্বয়ংক্রিয়ভাবে চারটি শ্রেণিতে ভাগ করতে হবে — `বাগ`, `ফিচার`, `প্রশ্ন`, আর `জরুরি`। প্রথমে zero-shot চেষ্টা করি:

```
# zero-shot — শুধু বলে দেওয়া
prompt = "এই ইমেইলটি শ্রেণীবদ্ধ করো: 'লগইন বাটনে ক্লিক করলে কিছু হয় না।'"
# সমস্যা: মডেল হয়তো বলবে "Bug report", কখনো "issue",
# কখনো "সমস্যা" — লেবেল ধারাবাহিক থাকে না।
```

কোড ৫.১ — উদাহরণ ছাড়া লেবেল এলোমেলো হয়ে যায়।

এবার few-shot — কয়েকটা নমুনা দিয়ে ঠিক লেবেল-সেট ও ফরম্যাট দেখিয়ে দিই:

```
messages = [
    {"role": "system", "content":
     "ইমেইলকে এই চারটির একটিতে ফেলো: বাগ, ফিচার, প্রশ্ন, জরুরি।"},

    # --- few-shot উদাহরণ ---
    {"role": "user", "content": "অ্যাপটা খুলতেই ক্র্যাশ করছে!"},
    {"role": "assistant", "content": "জরুরি"},

    {"role": "user", "content": "ডার্ক মোড যোগ করা যায় কি?"},
    {"role": "assistant", "content": "ফিচার"},

    {"role": "user", "content": "রিপোর্ট কীভাবে এক্সপোর্ট করব?"},
    {"role": "assistant", "content": "প্রশ্ন"},

    # --- আসল প্রশ্ন ---
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
    {"role": "user", "content": "লগইন বাটনে ক্লিক করলে কিছু হয় না।"}
]
# ফলাফল: "বাগ" — ধারাবাহিক, পরিষ্কার, ব্যবহারযোগ্য।
```

কোড ৫.২ — উদাহরণ দিয়ে মডেলকে ঠিক লেবেল-সেট ও ফরম্যাট শেখানো হলো।

লক্ষ্য করুন, উদাহরণগুলো চারটি ভিন্ন শ্রেণি ছুঁয়েছে — তাই মডেল ভারসাম্য বোঝে। আর প্রতিটা উত্তর ঠিক এক শব্দে, যাতে মডেলও এক শব্দেই উত্তর দেয়। এই ধারাবাহিকতাই few-shot-এর আসল শক্তি।

মনে রাখুন

Few-shot শুধু "কী করতে হবে" নয়, "কেমন করে দেখাতে হবে" সেটাও শেখায়। এই কারণেই এটা output format নিয়ন্ত্রণের সাথে গভীরভাবে যুক্ত — যা আমরা পরের অধ্যায়ে দেখব।

কী শিখলেন

- ✓ Zero-shot = উদাহরণ ছাড়া; few-shot = কয়েকটা নমুনা দেখিয়ে কাজ করানো।
- ✓ নির্দিষ্ট ফরম্যাট, টোন বা শ্রেণীবিন্যাসে few-shot বেশি কার্যকর।
- ✓ সংখ্যা নয়, মান-ই আসল — ২-৫টা প্রতিনিধিত্বমূলক উদাহরণই যথেষ্ট।
- ✓ শ্রেণিগুলোর ভারসাম্য না থাকলে মডেল একদিকে ঝুঁকে পড়ে।
- ✓ উদাহরণের ফরম্যাট হুবহু কপি করে মডেল — তাই ধারাবাহিকতা জরুরি।

পরের অধ্যায়ে আমরা শিখব *output format* নিয়ন্ত্রণ — কীভাবে মডেল থেকে নির্ভরযোগ্য *JSON* বা কাঠামোবদ্ধ উত্তর আদায় করতে হয়, যা আপনার কোড সরাসরি ব্যবহার করতে পারে।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৬

Output Format নিয়ন্ত্রণ

বাস্তব অ্যাপে আপনি শুধু "উত্তর" চান না — আপনি এমন উত্তর চান যা আপনার কোড নির্ভরযোগ্যভাবে ব্যবহার করতে পারে। এই অধ্যায়ে আমরা শিখব কীভাবে মডেল থেকে কাঠামোবদ্ধ, অনুমানযোগ্য আউটপুট আদায় করতে হয় — আর সেটা নিরাপদে প্রসেস করতে হয়।

৬.১ JSON ও structured output আদায় করা

একটা চ্যাটবটের উত্তর মুক্ত লেখা হতে পারে, সমস্যা নেই। কিন্তু আপনি যদি মডেলের আউটপুট কোনো প্রোগ্রামে পাঠান — ডেটাবেসে, UI-তে, বা পরের ধাপে — তখন আপনার দরকার নির্দিষ্ট ক্ষেত্র (field) সহ অনুমানযোগ্য কাঠামো, একটা প্যারাগ্রাফ নয়।

এর সবচেয়ে প্রচলিত উপায় **JSON**। কৌশলটা সহজ — মডেলকে স্পষ্টভাবে কাঠামোটা বলুন, আর জোর দিয়ে বলুন শুধু JSON-ই ফেরত দিতে, বাড়তি কোনো লেখা বা markdown ছাড়া।

```
# TaskFlow: ইমেইল থেকে কাঠামোবদ্ধ তথ্য বের করা
SYSTEM = """ইমেইল বিশ্লেষণ করে শুধু নিচের JSON ফেরত দাও,
অন্য কোনো লেখা বা markdown ছাড়া:
{"category": "বাগ|ফিচার|প্রশ্ন|জরুরি",
 "priority": "low|medium|high",
 "summary": "এক বাক্যে সারাংশ"}"""

# প্রত্যাশিত আউটপুট:
# {"category": "বাগ", "priority": "high",
#  "summary": "লগইন বাটন কাজ করছে না।"}
```

কোড ৬.১ — স্পষ্ট schema দিলে মডেল ব্যবহারযোগ্য JSON ফেরত দেয়।

> **মনে রাখুন**
>
> অনেক প্রোভাইডারের একটা "JSON mode" বা structured output ফিচার থাকে যা বৈধ JSON নিশ্চিত করে। সম্ভব হলে সেটা ব্যবহার করুন — হাতে পার্স করার ঝুঁকি কমে।

Licensed to sakilorb@gmail.com · Do not redistribute

৬.২ XML ট্যাগ দিয়ে নির্ভরযোগ্যতা বাড়ানো

আউটপুটে যখন একসাথে মুক্ত লেখা আর কাঠামোবদ্ধ অংশ — দুটোই থাকে, তখন XML-ধরনের ট্যাগ দারুণ কাজ করে। মডেলকে বলুন আলাদা আলাদা অংশ আলাদা ট্যাগে রাখতে, পরে আপনি শুধু দরকারি অংশটা টেনে নিন।

ShopBondhu-র একটা সুন্দর প্রয়োগ দেখুন — বট ক্রেতার জন্য একটা উত্তর দেবে, পাশাপাশি আপনার সিস্টেমের জন্য একটা সংকেত দেবে যে সমস্যাটা মানুষ-এজেন্টের কাছে পাঠানো দরকার কিনা:

```
# মডেলকে দুই অংশে আউটপুট দিতে বলা
SYSTEM = """এভাবে উত্তর দাও:
<reply>ক্রেতার জন্য বাংলা উত্তর</reply>
<escalate>true অথবা false</escalate>"""

# মডেলের আউটপুট:
# <reply>দুঃখিত! আমি বিষয়টি আমাদের টিমকে জানাচ্ছি।</reply>
# <escalate>true</escalate>

reply = extract_tag(output, "reply")      # ক্রেতাকে দেখাই
if extract_tag(output, "escalate") == "true":
    route_to_human()                      # সিস্টেম সিদ্ধান্ত নেয়
```

কোড ৬.২ — ট্যাগ দিয়ে ক্রেতার উত্তর ও সিস্টেম-সংকেত আলাদা রাখা।

ট্যাগের আরেকটা সুবিধা — উত্তরে যখন উদ্ধৃতি বা বিশেষ অক্ষর থাকে যা JSON ভেঙে দিতে পারে, তখন ট্যাগ অনেক বেশি সহনশীল। আর মনে রাখুন, ট্যাগ শুধু আউটপুটে নয় — ইনপুট কন্টেক্সট সাজাতেও দারুণ; ডকুমেন্ট বা ডেটা ট্যাগে মুড়ে দিলে মডেল সীমানা স্পষ্ট বোঝে।

৬.৩ নিরাপদে পার্সিং ও ভ্যালিডেশন

একটা কঠিন সত্য — মডেল ১০০% নির্ভরযোগ্য নয়। কখনো সে বাড়তি লেখা জুড়ে দেয়, কখনো markdown fence (```) মুড়ে দেয়, কখনো সামান্য ভাঙা JSON দেয়। তাই আপনার কোডকে এসব সামলাতে প্রস্তুত থাকতে হবে — কখনোই অন্ধভাবে বিশ্বাস করা যাবে না।

```
import json

def safe_parse(text):
    # ১) markdown fence থাকলে ছেঁটে ফেলি
    text = text.replace("```json", "").replace("```", "").strip()
    try:
        data = json.loads(text)
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
    except json.JSONDecodeError:
        return None    # ভাঙা হলে নিরাপদে ব্যর্থ হই

    # ২) প্রত্যাশিত ক্ষেত্র ও মান যাচাই
    valid = {"বাগ", "ফিচার", "প্রশ্ন", "জরুরি"}
    if data.get("category") not in valid:
        return None
    return data
```

কোড ৬.৩ — fence ছাঁটা, try/except, আর মান যাচাই — তিন স্তরের সুরক্ষা।

সাবধান

পার্স ব্যর্থ হলে অ্যাপ যেন ক্র্যাশ না করে। একটা নিরাপদ ডিফল্ট রাখুন, অথবা মডেলকে আবার চেষ্টা করতে দিন (retry)। প্রোডাকশনে এই সুরক্ষাটাই আপনাকে রাতে শান্তিতে ঘুমাতে দেবে।

কী শিখলেন

- ✓ কোড যখন আউটপুট ব্যবহার করবে, তখন প্রোজ নয় — কাঠামোবদ্ধ আউটপুট দরকার।
- ✓ স্পষ্ট schema দিয়ে এবং "শুধু JSON" বলে নির্ভরযোগ্য JSON আদায় করা যায়।
- ✓ XML ট্যাগ মুক্তলেখা ও সংকেত আলাদা রাখতে এবং ইনপুট সাজাতে দারুণ।
- ✓ মডেল নির্ভুল নয় — fence ছাঁটা, try/except ও ভ্যালিডেশন আবশ্যক।
- ✓ পার্স ব্যর্থ হলে নিরাপদ ডিফল্ট বা retry রাখুন; অ্যাপ যেন না ভাঙে।

এই অধ্যায়ের সাথে শেষ হলো পর্ব ২। এতক্ষণ আমরা স্থির কন্টেক্সট নিয়ে কাজ করেছি। পরের পর্বে আমরা কন্টেক্সটকে জীবন্ত করব — বাইরের ডেটা, স্মৃতি ও টুলের সাথে সংযোগ ঘটিয়ে, শুরু হবে *RAG* দিয়ে।

Licensed to sakilorb@gmail.com · Do not redistribute

পর্ব ৩

বাইরের জগতের সাথে সংযোগ

এতক্ষণ কন্টেক্সট ছিল স্থির — আমরা হাতে তথ্য সাজিয়ে দিয়েছি। এই পর্বে কন্টেক্সট হবে জীবন্ত ও গতিশীল। মডেল বাইরের ডেটা টেনে আনবে (RAG), আগের কথা মনে রাখবে (memory), আর নিজে কাজ করবে (tools)।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৭

RAG — প্রাসঙ্গিক ডেটা টেনে আনা

একটা মডেল সব জানে না — বিশেষ করে আপনার নিজের ডেটা, যেমন আপনার পণ্য বা ডকুমেন্টসে জানে না। *RAG* হলো সেই কৌশল যেখানে প্রশ্নের সাথে প্রাসঙ্গিক তথ্য খুঁজে এনে কন্টেক্সটে যোগ করা হয়। এই অধ্যায়ে আমরা সেটা গোড়া থেকে বুঝব।

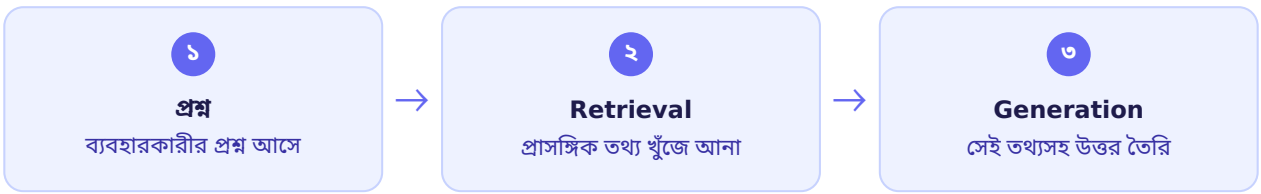
৭.১ RAG কী ও কেন দরকার

RAG-এর পুরো নাম Retrieval-Augmented Generation — অর্থাৎ Retrieval (খুঁজে আনা) আর Generation (উত্তর তৈরি) মিলিয়ে। সমস্যাটা সহজ: মডেল আপনার নিজস্ব বা সাম্প্রতিক ডেটা চেনে না, আর সব ডেটা একসাথে কন্টেক্সটে দেওয়াও যায় না (মনে আছে অধ্যায় ৩-এর context rot?)।

সমাধান — আপনার জ্ঞানভাণ্ডার বাইরে আলাদা করে রাখুন। প্রশ্ন এলে শুধু প্রাসঙ্গিক অংশটুকু খুঁজে এনে কন্টেক্সটে দিন, তারপর মডেলকে উত্তর তৈরি করতে দিন।

> **উপমা**
>
> RAG অনেকটা ওপেন-বুক পরীক্ষার মতো। মডেল একজন মেধাবী ছাত্র; RAG তাকে উত্তর দেওয়ার আগে বইয়ের ঠিক পাতাটা খুলে দেখতে দেয় — স্মৃতির ওপর ভরসা করতে হয় না।

এতে তিনটা বড় সমস্যা একসাথে মেটে — উত্তর আসল ডেটায় ভিত্তি করে হয় বলে hallucination কমে; শুধু প্রাসঙ্গিক অংশ আসে বলে context-সীমা সমস্যা হয় না; আর জ্ঞানভাণ্ডার আপডেট করলেই তথ্য নতুন থাকে, মডেল বদলাতে হয় না। পুরো প্রক্রিয়াটা তিন ধাপের:



৭.২ Embedding ও vector search সহজ ভাষায়

প্রশ্ন এলে "প্রাসঙ্গিক তথ্য" খুঁজব কীভাবে? শুধু কীওয়ার্ড মেলানো দুর্বল উপায় — কারণ একই অর্থ ভিন্ন শব্দে আসতে পারে। এখানেই আসে **embedding**।

Licensed to sakilorb@gmail.com · Do not redistribute

Embedding মানে একটা লেখাকে একগুচ্ছ সংখ্যায় (একটা vector-এ) রূপান্তর করা, যা ওই লেখার অর্থ ধরে রাখে। কাছাকাছি অর্থের লেখার vector কাছাকাছি হয়।

> **উপমা**
>
> ভাবুন একটা ম্যাপ, যেখানে কাছাকাছি বিষয়গুলো কাছাকাছি বসানো। "ফোন" আর "মোবাইল" সেখানে পাশাপাশি পড়বে, যদিও শব্দ দুটো আলাদা। Embedding ঠিক এই ম্যাপটাই তৈরি করে।

এখন **vector search** সহজ — প্রশ্নটাকেও embed করুন, তারপর সঞ্চিত vector-গুলোর মধ্যে সবচেয়ে কাছেরগুলো বেছে নিন; সেগুলোই সবচেয়ে প্রাসঙ্গিক। এই গণিত হাতে বানাতে হয় না — embedding মডেল আর vector database (যেমন FAISS, pgvector, Pinecone) কাজটা করে দেয়।

```
# ধারণাগত vector search
query_vec = embed("১৫ হাজার টাকার হেডফোন")

# সঞ্চিত পণ্যের সাথে মিল বের করে কাছেরগুলো নেওয়া
results = vector_db.search(query_vec, top_k=3)
# results = সবচেয়ে প্রাসঙ্গিক ৩টি পণ্য
```

কোড ৭.১ — প্রশ্ন embed করে কাছের তথ্য খুঁজে আনা।

৭.৩ Chunking কৌশল

একটা ৫০ পৃষ্ঠার ডকুমেন্টকে একটা মাত্র vector বানালে তা বড্ড স্থূল হয়ে যায় — খুঁজে নির্দিষ্ট অংশ পাওয়া যায় না। তাই বড় লেখাকে ছোট ছোট টুকরোয় ভাগ করতে হয়, প্রতিটা টুকরোকে বলে **chunk**।

ভালো chunking-এর ভারসাম্যটা জরুরি — খুব বড় chunk নির্ভুলতা হারায় ও টোকেন নষ্ট করে, খুব ছোট chunk অর্থ হারিয়ে ফেলে। তাই অর্থপূর্ণ একক একসাথে রাখুন (বাক্য বা ধারণার মাঝখানে কাটবেন না), আর সীমানায় একটু overlap রাখুন যাতে প্রসঙ্গ হারিয়ে না যায়।

> **মনে রাখুন**
>
> সম্ভব হলে কাঠামো অনুযায়ী ভাগ করুন, অন্ধভাবে অক্ষর গুনে নয়। ShopBondhu-তে প্রতিটা পণ্য = একটা chunk; প্রতিটা পলিসি-অনুচ্ছেদ = একটা chunk। এতে প্রতিটা টুকরো নিজে থেকেই অর্থপূর্ণ।

Licensed to sakilorb@gmail.com · Do not redistribute

৭.৪ ShopBondhu প্রোডাক্ট ক্যাটালগ দিয়ে RAG

এবার সব মিলিয়ে দেখি। ShopBondhu-তে ৫,০০০ পণ্য আছে। একজন ক্রেতা জিজ্ঞেস করল — "১৫,০০০ টাকার মধ্যে ভালো হেডফোন আছে?" পুরো ক্যাটালগ কন্টেক্সটে ঢাললে কী হবে, তা তো আমরা অধ্যায় ৩-এ দেখেছি — ধীর, ব্যয়বহুল, আর ভুলে ভরা। RAG দিয়ে সমাধান:

```
# ১) আগে থেকে প্রতিটা পণ্য embed করে রাখা (index)
for p in products:
    vector_db.add(embed(p.description), meta=p)

# ২) প্রশ্ন এলে শুধু প্রাসঙ্গিক ৩টি পণ্য টেনে আনা
hits = vector_db.search(embed(user_msg), top_k=3)

# ৩) সেই ৩টি পণ্যসহ কন্টেক্সট বানিয়ে উত্তর তৈরি
context = "\n".join(h.meta.as_text() for h in hits)
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "system", "content": f"প্রাসঙ্গিক পণ্য:\n{context}"},
    {"role": "user", "content": user_msg}
]
```

কোড ৭.২ — ৫,০০০ পণ্যের বদলে শুধু প্রাসঙ্গিক ৩টি কন্টেক্সটে।

ফলাফল — সঠিক, বাস্তব ডেটায় ভিত্তি করা, আর সাশ্রয়ী উত্তর। তবে একটা কথা মনে রাখবেন:

> **সাবধান**
>
> RAG-এর মান পুরোপুরি নির্ভর করে retrieval-এর মানের ওপর। ভুল তথ্য টেনে আনলে উত্তরও ভুল হবে — "garbage in, garbage out"। তাই chunking ও search ভালো করে যাচাই করুন।

একটা বাস্তব retrieval দৃশ্য। ব্যাপারটা আরও পরিষ্কার করতে দেখি — ক্রেতা যখন "১৫,০০০ টাকার মধ্যে ভালো হেডফোন আছে?" জিজ্ঞেস করে, vector search ৫,০০০ পণ্য থেকে ঠিক কী টেনে আনে:

```
# ক্রেতার প্রশ্ন embed করে কাছের ৩টি পাওয়া গেল:
[
  {"name": "SoundPro X2 হেডফোন", "price": 12500,
   "score": 0.91},   # সবচেয়ে প্রাসঙ্গিক
  {"name": "BassBuds Pro",      "price": 9800,
   "score": 0.87},
  {"name": "AirZite 300",        "price": 14200,
   "score": 0.84}
]
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
# লক্ষ্য করুন: "১৫ হাজার" না লিখলেও দামের সীমা বোঝা গেছে,
# আর "ইয়ারফোন/হেডফোন" অর্থগত মিলেই এসেছে — কীওয়ার্ড নয়।
```

কোড ৭.৩ — embedding অর্থ বোঝে বলেই প্রাসঙ্গিক ৩টি পণ্য উঠে এসেছে।

এই ৩টি পণ্যই কেবল কন্টেক্সটে যাবে — ৫,০০০ নয়। মডেল এবার আসল দাম ও তথ্য দেখে সঠিক সুপারিশ দিতে পারবে, hallucination ছাড়াই।

TaskFlow-এও RAG দরকার। একটা টিমের হয়তো শত শত প্রজেক্ট আর হাজারো টাস্ক আছে। কেউ যখন জিজ্ঞেস করে "পেমেন্ট মডিউল নিয়ে আগের সিদ্ধান্তগুলো কী ছিল?", তখন সব টাস্ক কন্টেক্সটে না দিয়ে — শুধু "পেমেন্ট"সম্পর্কিত প্রাসঙ্গিক টাস্ক ও কমেন্টগুলো embed-search দিয়ে টেনে আনা হয়। নীতিটা ShopBondhu-র মতোই; শুধু ডেটা পণ্যের বদলে টাস্ক।

অনুশীলন

নিজে চেষ্টা করুন — ShopBondhu-র জন্য একটা ছোট RAG ভাবুন:

1. আপনার ১০টি পণ্যের প্রতিটিকে একটা chunk হিসেবে লিখুন (নাম, দাম, বর্ণনা)।
2. "বাজেট ফোন কোনটা?" — এই প্রশ্নে কোন ২-৩টি chunk আসা উচিত, হাতে বেছে নিন।
3. ভাবুন: যদি কীওয়ার্ড মেলানো ব্যবহার করতেন, কোথায় ভুল হতো?

কী শিখলেন

- ✓ RAG = প্রাসঙ্গিক তথ্য খুঁজে এনে (retrieval) তারপর উত্তর তৈরি (generation)।
- ✓ এটি hallucination কমায়, context-সীমা সামলায়, আর তথ্য নতুন রাখে।
- ✓ Embedding লেখাকে অর্থবাহী vector-এ বদলায়; কাছের vector = প্রাসঙ্গিক।
- ✓ বড় লেখা অর্থপূর্ণ chunk-এ ভাগ করতে হয় — খুব বড়ও নয়, খুব ছোটও নয়।
- ✓ retrieval-এর মানই RAG-এর মান ঠিক করে — ভুল তথ্য মানে ভুল উত্তর।

পরের অধ্যায়ে আমরা শিখব *conversation history* ও *memory* — কীভাবে মডেল একাধিক ধাপের কথোপকথন মনে রাখে, আর সেই ইতিহাস ছোট ও সাশ্রয়ী রাখা যায়।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৮

Conversation History ও Memory

অধ্যায় ১-এ আমরা জেনেছি মডেলের নিজস্ব স্মৃতি নেই। তাহলে চ্যাটবট আগের কথা "মনে রাখে" কীভাবে? এই অধ্যায়ে আমরা শিখব মাল্টি-টার্ন কথোপকথন সামলানো, ইতিহাস ছোট রাখা, আর দীর্ঘমেয়াদি *memory* ডিজাইন।

৮.১ মাল্টি-টার্ন কথোপকথন সামলানো

যেহেতু মডেল প্রতিবার নতুন করে শুরু করে, একটা চলমান কথোপকথন রাখতে হলে কৌশলটা একটাই — **আপনি** ইতিহাস জমিয়ে রাখুন, আর প্রতিবার পুরো ইতিহাসটা আবার পাঠান। সহজ কথায়, `messages` তালিকাটাই মডেলের স্মৃতি।

প্রতি ধাপে ব্যবহারকারীর মেসেজ আর মডেলের উত্তর — দুটোই তালিকায় যোগ করুন, তারপর পরের বার পুরো তালিকা পাঠান:

```
messages = [{"role": "system", "content": SYSTEM_PROMPT}]

def chat(user_msg):
    messages.append({"role": "user", "content": user_msg})
    reply = call_model(messages)            # পুরো ইতিহাসসহ
    messages.append({"role": "assistant", "content": reply})
    return reply

# প্রতিবার পুরো messages পাঠানো হয় বলেই
# মডেল আগের কথা "মনে রাখে"।
```

কোড ৮.১ — ইতিহাস জমিয়ে প্রতিবার আবার পাঠানোই মাল্টি-টার্নের রহস্য।

> **সাবধান**
>
> ইতিহাস আবার না পাঠালে মডেল দ্বিতীয় ধাপেই প্রথম ধাপের কথা ভুলে যাবে। "আগে তো বলেছিলাম" — এই অভিযোগ মডেলের কাছে অর্থহীন।

Licensed to sakilorb@gmail.com · Do not redistribute

৮.২ হিস্ট্রি ছোট রাখার কৌশল (summarization)

কিন্তু এখানে একটা সমস্যা লুকিয়ে আছে — কথোপকথন যত দীর্ঘ হয়, ইতিহাসও তত বড় হয়। এক সময় তা context-সীমা ছাড়িয়ে যায়, খরচ বাড়ায়, আর গতি কমায় (আবারও অধ্যায় ৩-এর হিসাব)। এই বাড়ন্ত ইতিহাস সামলানোর কয়েকটা কৌশল আছে:

ইতিহাস ছোট রাখার ৩টি কৌশল

১. **Sliding window** — শুধু শেষ কয়েকটা ধাপ রাখা। সহজ, তবে পুরোনো প্রসঙ্গ হারায়।

২. **Summarization** — পুরোনো ধাপগুলোকে ছোট সারাংশে চেপে, সাম্প্রতিক ধাপগুলো হুবহু রাখা। দুই জগতের সেরা।

৩. **গুরুত্ব অনুযায়ী বাছাই** — দরকারি তথ্য রাখা, অপ্রয়োজনীয় খুচরো কথা ফেলে দেওয়া।

সবচেয়ে কার্যকর সাধারণত summarization। ইতিহাস বড় হয়ে গেলে পুরোনো অংশটা মডেল দিয়েই সংক্ষেপ করিয়ে নিন, তারপর অনেকগুলো মেসেজের বদলে একটা সারাংশ-মেসেজ রাখুন:

```
if token_count(messages) > LIMIT:
    old = messages[1:-6]              # পুরোনো অংশ
    summary = call_model([
        {"role": "user",
         "content": f"সংক্ষেপে সার বলো:\n{old}"}
    ])
    # অনেক মেসেজ → একটা সারাংশ
    messages = (messages[:1]
        + [{"role": "system", "content": f"আগের আলোচনার সার: {summary}"}]
        + messages[-6:])      # সাম্প্রতিক ৬টি হুবহু
```

কোড ৮.২ — পুরোনো ইতিহাস চেপে, সাম্প্রতিক অংশ হুবহু রাখা।

TaskFlow-এর একটা দীর্ঘ পরিকল্পনা-আলাপে এই কৌশল দারুণ কাজ করে — শুরুর সিদ্ধান্তগুলো এক প্যারায় সার করে রাখলে আলোচনার সূত্র হারায় না, অথচ ইতিহাস হালকা থাকে।

৮.৩ দীর্ঘমেয়াদি memory ডিজাইন

এতক্ষণের ইতিহাস হলো **স্বল্পমেয়াদি স্মৃতি** — এক সেশনের মধ্যে সীমাবদ্ধ। কিন্তু একটা ভালো সহকারী সেশন পেরিয়েও মনে রাখে — যেমন ফিরে আসা ক্রেতার নাম, পছন্দ, আগের অর্ডার। একে বলে **দীর্ঘমেয়াদি memory**।

Licensed to sakilorb@gmail.com · Do not redistribute

এর কৌশলটা আসলে RAG-এরই একটা রূপ (অধ্যায় ৭ মনে আছে?) — ব্যবহারকারী সম্পর্কে স্থায়ী তথ্য বাইরের একটা store-এ রাখুন, আর দরকারমতো প্রাসঙ্গিকগুলো কন্টেক্সটে টেনে আনুন।

এক সেশন

স্বল্পমেয়াদি

- messages তালিকায় থাকে
- প্রতিবার পাঠানো হয়
- সেশন শেষে মুছে যায়
- সাম্প্রতিক প্রসঙ্গ ধরে

সেশন পেরিয়ে

দীর্ঘমেয়াদি

- বাইরের store/DB-তে থাকে
- দরকারমতো টেনে আনা হয়
- স্থায়ীভাবে থাকে
- স্থায়ী তথ্য/পছন্দ ধরে

মনে রাখুন

সব মেসেজ নয় — শুধু টেকসই তথ্য সংরক্ষণ করুন (নাম, পছন্দ, গুরুত্বপূর্ণ সিদ্ধান্ত)। আর memory-তেও context rot খাটে: যত বেশি অপ্রাসঙ্গিক স্মৃতি, তত বেশি বিভ্রান্তি। সংবেদনশীল তথ্য সংরক্ষণে সতর্ক থাকুন।

কী শিখলেন

- ✓ মডেল stateless — আপনি ইতিহাস জমিয়ে প্রতিবার আবার পাঠান।
- ✓ messages তালিকাই কথোপকথনের স্বল্পমেয়াদি স্মৃতি।
- ✓ ইতিহাস বড় হলে sliding window বা summarization দিয়ে ছোট রাখুন।
- ✓ দীর্ঘমেয়াদি memory = বাইরের store-এ স্থায়ী তথ্য + দরকারমতো retrieval।
- ✓ শুধু টেকসই, প্রাসঙ্গিক তথ্য রাখুন; সংবেদনশীল তথ্যে সতর্ক থাকুন।

পরের অধ্যায়ে আমরা মডেলকে *"হাত-পা" দেব — tools ও function calling,* যাতে মডেল শুধু কথা না বলে আসল কাজও করতে পারে, যেমন অর্ডার খোঁজা বা রিফান্ড প্রসেস করা।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ০৯

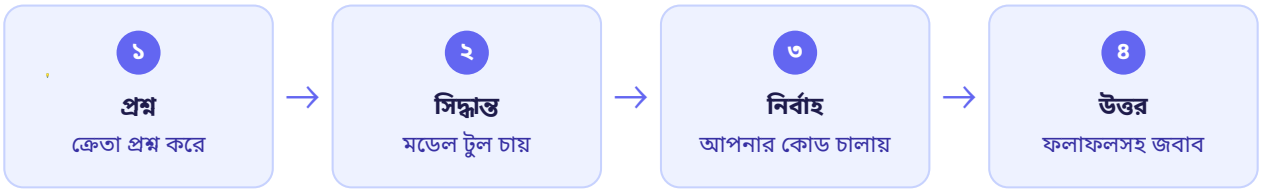
Tools ও Function Calling

এতক্ষণ মডেল শুধু কথা বলেছে। কিন্তু আসল সহকারী কাজও করে — অর্ডার খোঁজে, রিফান্ড প্রসেস করে, টাস্ক বানায়। *Tools* বা *function calling* মডেলকে "হাত-পা" দেয়, যাতে সে আপনার ফাংশন ডেকে বাস্তবে কাজ করতে পারে।

৯.১ মডেলকে "হাত-পা" দেওয়া

এতদিন মডেল ছিল শুধু একটা মস্তিষ্ক যা কেবল লেখা তৈরি করে। Tools সেই মস্তিষ্ককে বাস্তব কাজ স্থির করার ক্ষমতা দেয়। তবে এখানে একটা সূক্ষ্ম কিন্তু গুরুত্বপূর্ণ ব্যাপার আছে — **মডেল নিজে ফাংশন চালায় না।**

মডেল শুধু বলে "এই টুলটা, এই আর্গুমেন্ট দিয়ে চালাও দরকার।" তারপর **আপনার কোড** সেটা চালায়, ফলাফল মডেলকে ফেরত দেয়, আর মডেল সেই ফলাফল দিয়ে চূড়ান্ত উত্তর সাজায়। অর্থাৎ মডেল সিদ্ধান্ত নেয়, আপনার কোড নির্বাহ করে।



মনে রাখুন

মডেল কখনো সরাসরি আপনার ডেটাবেস ছোঁয় না। সে শুধু অনুরোধ করে; আসল নিয়ন্ত্রণ সবসময় আপনার কোডের হাতে থাকে। এটাই নিরাপত্তার ভিত্তি।

৯.২ টুল সংজ্ঞা (schema) লেখা

মডেল কোনো টুল ব্যবহার করতে হলে আগে জানতে হবে — টুলটা আছে, আর কীভাবে ডাকতে হয়। এজন্য আপনি একটা **schema** দেন: টুলের নাম, কাজের বর্ণনা, আর প্যারামিটারগুলো (টাইপ ও বর্ণনাসহ)।

এখানে একটা চমৎকার ব্যাপার — টুলের বর্ণনাটাও কিন্তু context engineering! পরিষ্কার বর্ণনা দিলে মডেল ঠিকঠাক টুল ব্যবহার করে; অস্পষ্ট বর্ণনা দিলে ভুল করে।

Licensed to sakilorb@gmail.com · Do not redistribute

```
tool = {
    "name": "get_order_status",
    "description": "একটি অর্ডারের বর্তমান অবস্থা ও "
                   "ডেলিভারি তারিখ ফেরত দেয়।",
    "parameters": {
        "order_id": {
            "type": "string",
            "description": "অর্ডার নম্বর, যেমন SB-2043"
        }
    }
}
```

কোড ৯.১ — পরিষ্কার নাম, বর্ণনা ও প্যারামিটার = সঠিক ব্যবহার।

৯.৩ ShopBondhu: order_status ও refund টুল

এবার ShopBondhu-তে দুটো টুল দিই — `get_order_status` আর `request_refund`। ক্রেতা যখন জিজ্ঞেস করে "আমার #SB-2043 অর্ডারটা কোথায়?", পুরো চক্রটা এমন হয়:

```
# মডেল টুল চাইলে আমাদের কোড সেটা চালায়
def handle(model_request):
    if model_request.tool == "get_order_status":
        oid = model_request.args["order_id"]
        result = db.get_order(oid)        # আসল DB কল
        return result                     # মডেলকে ফেরত

    if model_request.tool == "request_refund":
        return safe_refund(model_request.args)  # সতর্ক পথ

# মডেল ফলাফল পেয়ে বলে:
# "আপনার #SB-2043 অর্ডারটি ১৮ জুন ডেলিভারি হবে।"
```

কোড ৯.২ — মডেল টুল চায়, আপনার কোড চালায়, মডেল উত্তর সাজায়।

খেয়াল করুন, `request_refund` একটা সংবেদনশীল কাজ — টাকা ফেরত। এমন টুল সরাসরি চালানো বিপজ্জনক, তাই আমরা একটা সতর্ক পথে (`safe_refund`) পাঠিয়েছি। কেন — সেটাই পরের অংশ।

৯.৪ নিরাপত্তা ও সীমা নির্ধারণ

Tools মানে ক্ষমতা, আর ক্ষমতা মানে ঝুঁকি। মডেল এখন বাস্তব কাজ—টাকা ফেরত করতে পারে, তাই টুলগুলো পাহারা দেওয়া জরুরি। কয়েকটা মূলনীতি:

Licensed to sakilorb@gmail.com · Do not redistribute

টুল নিরাপত্তার ৫টি মূলনীতি

১. **আর্গুমেন্ট যাচাই করুন** — মডেলের আউটপুট অন্ধভাবে বিশ্বাস নয়; চালানোর আগে আপনার কোডে যাচাই করুন।

২. **সর্বনিম্ন ক্ষমতা** — যতটুকু দরকার ঠিক ততটুকু টুল ও অনুমতি দিন, বেশি নয়।

৩. **অনুমোদন যাচাই** — ব্যবহারকারী কি এই কাজের অধিকারী? যেমন শুধু নিজের অর্ডারই রিফান্ড।

৪. **সীমা বসান** — যেমন নির্দিষ্ট অঙ্কের বেশি রিফান্ড স্বয়ংক্রিয় নয়।

৫. **মানুষ-ইন-দ্য-লুপ** — সংবেদনশীল কাজে মানুষের অনুমোদন রাখুন (মনে আছে অধ্যায় ৬-এর escalate?)।

```
def safe_refund(args):
    order = db.get_order(args["order_id"])
    if order.user_id != current_user.id:      # অনুমোদন
        return "এই অর্ডার আপনার নয়।"
    if order.amount > 5000:                   # সীমা
        return route_to_human(order)           # মানুষ-ইন-দ্য-লুপ
    return process_refund(order)              # নিরাপদ হলে তবেই
```

কোড ৯.৩ — অনুমোদন, সীমা ও হস্তান্তর — তিন স্তরের সুরক্ষা।

> **সাবধান**
>
> মডেলকে কখনো এমন টুল দেবেন না যা অপূরণীয় ক্ষতি করতে পারে — কোনো যাচাই ছাড়া। প্রতিটা সংবেদনশীল কাজের সিদ্ধান্ত শেষমেশ আপনার কোডের হাতে রাখুন, মডেলের হাতে নয়।

TaskFlow-এও একই কাঠামো। অভ্যন্তরীণ সহকারীকে একটা `create_task` টুল দিলে ব্যবহারকারী শুধু বললেই টাস্ক তৈরি হয়:

```
create_task = {
    "name": "create_task",
    "description": "একটি নতুন টাস্ক তৈরি করে নির্দিষ্ট প্রজেক্টে যোগ করে।",
    "parameters": {
        "project_id": {"type": "string"},
        "title":      {"type": "string", "description": "টাস্কের শিরোনাম"},
        "priority":   {"type": "string", "description": "low/medium/high"}
    }
}
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
# ব্যবহারকারী: "লগইন বাগটা high priority টাস্ক বানাও"
# → মডেল create_task চায়; আপনার কোড যাচাই করে টাস্কটি বানায়।
```

কোড ৯.৪ — TaskFlow-এর create_task: একই নিরাপত্তা-নীতি, ভিন্ন কাজ।

গ্রাহকমুখী রিফান্ড হোক বা অভ্যন্তরীণ টাস্ক তৈরি — নীতি এক: মডেল চায়, আপনার কোড যাচাই করে চালায়।

কী শিখলেন

- ✓ Tools মডেলকে বাস্তব কাজ করার ক্ষমতা দেয় (function calling)।
- ✓ মডেল টুল চায়, কিন্তু চালায় আপনার কোড — নিয়ন্ত্রণ আপনার হাতে।
- ✓ পরিষ্কার schema (নাম, বর্ণনা, প্যারামিটার) সঠিক ব্যবহারের চাবি।
- ✓ সংবেদনশীল টুলে আর্গুমেন্ট যাচাই, অনুমোদন ও সীমা আবশ্যক।
- ✓ ঝুঁকিপূর্ণ কাজে মানুষ-ইন-দ্য-লুপ রাখুন; মডেলকে অন্ধভাবে বিশ্বাস নয়।

এই অধ্যায়ের সাথে শেষ হলো পর্ব ৩। এখন আমাদের হাতে সব উপাদান আছে। শেষ পর্বে আমরা সব একসাথে জোড়া লাগাব — কন্টেক্সট অপটিমাইজ করা, আর শূন্য থেকে একটা পূর্ণ *ShopBondhu* চ্যাটবট গড়া।

Licensed to sakilorb@gmail.com · Do not redistribute

পর্ব ৪

সব একসাথে — পেশাদারি কৌশল

সব উপাদান হাতে চলে এসেছে। এই শেষ পর্বে আমরা শিখব কীভাবে সব একসাথে জোড়া লাগাতে হয় — কন্টেক্সট অপটিমাইজ করা, আর শূন্য থেকে একটা পূর্ণ ShopBondhu চ্যাটবট গড়া। শেষে থাকবে একটা সেরা-অভ্যাসের চেকলিস্ট।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১০

Context সাজানো ও অপটিমাইজেশন

উপকরণ হাতে থাকা আর রান্না করা এক জিনিস নয়। এই অধ্যায়ে আমরা শিখব কন্টেক্সট ভালোভাবে সাজানো — ক্রম, প্রাসঙ্গিকতা বনাম পরিমাণ, আর টোকেন বাজেট — যাতে আপনার অ্যাপ দ্রুত, সাশ্রয়ী আর নির্ভুল হয়।

১০.১ ক্রম (ordering) কেন গুরুত্বপূর্ণ

অধ্যায় ৩-এর "lost in the middle" মনে আছে? মডেল কন্টেক্সটের শুরু আর শেষে বেশি মনোযোগ দেয়, মাঝখানে কম। তাই কোন তথ্য কোথায় রাখছেন, সেটা সরাসরি উত্তরের মানে প্রভাব ফেলে।

একটা নির্ভরযোগ্য সাজানোর ছক:

কন্টেক্সট সাজানোর আদর্শ ক্রম

১. **System prompt ও মূল নিয়ম** — সবার উপরে, যেখানে মনোযোগ বেশি।
২. **প্রাসঙ্গিক ডেটা** (RAG থেকে আনা) — নিয়মের পরে।
৩. **উদাহরণ** (few-shot) — ধরন বোঝাতে।
৪. **কথোপকথনের ইতিহাস** — সংক্ষিপ্ত আকারে।
৫. **বর্তমান প্রশ্ন** — সবার শেষে, যাতে সবচেয়ে তাজা থাকে।

প্রশ্নটা শেষে রাখার কারণ — সেটাই মডেলের মনোযোগে সবচেয়ে তাজা থাকে, আর মডেল সাম্প্রতিকতম জিনিসেরই উত্তর দেয়।

১০.২ প্রাসঙ্গিকতা বনাম শব্দসংখ্যা

আবারও সেই সোনালি নিয়ম — প্রাসঙ্গিকতা > পরিমাণ। আপনি কন্টেক্সটে যত টোকেন যোগ করেন, প্রতিটাকে তার জায়গা অর্জন করতে হবে। "দরকার হলে কাজে লাগবে" ভেবে বাড়তি তথ্য ঢালা ক্ষতিই করে — context rot বাড়ে খরচ বাড়ে আর গুরুত্বপূর্ণ তথ্য মাঝখানে হারিয়ে যায়।

একটা নিষ্ঠুর প্রশ্ন নিজেকে করুন: "এই অংশটা সরিয়ে দিলে কি উত্তর বদলাবে?" না হলে, বাদ দিন।

Licensed to sakilorb@gmail.com · Do not redistribute

দুর্বল

ফোলানো কন্টেক্সট

- ২০টি পণ্য "নিরাপত্তার জন্য"
- পুরো পলিসি ডকুমেন্ট
- সম্পূর্ণ চ্যাট ইতিহাস
- ~২০,০০০ টোকেন, ধীর, ভুলে ভরা

শক্তিশালী

মনোযোগী কন্টেক্সট

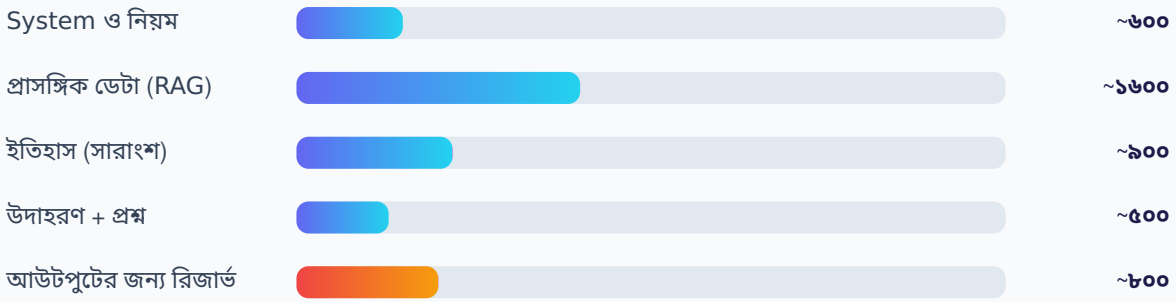
- প্রাসঙ্গিক ৩টি পণ্য
- শুধু সংশ্লিষ্ট পলিসি-অনুচ্ছেদ
- সারাংশ + সাম্প্রতিক ধাপ
- ~২,০০০ টোকেন, দ্রুত, নিখুঁত

১০.৩ টোকেন বাজেট সামলানো

কন্টেক্সটকে একটা নির্দিষ্ট বাজেট হিসেবে ভাবুন। সেই বাজেট সচেতনভাবে উপাদানগুলোর মধ্যে ভাগ করুন — কোনটায় কত টোকেন যাবে, আগে থেকে ঠিক করুন। আর মনে রাখবেন, আউটপুটও টোকেন খরচ করে (অধ্যায় ৩), তাই উত্তরের জন্যও জায়গা রাখুন।

একটি নমুনা টোকেন বাজেট

ShopBondhu বট — উপাদান অনুযায়ী আনুমানিক বরাদ্দ



বাজেট ছাড়িয়ে গেলে সবচেয়ে কম-গুরুত্বপূর্ণ অংশটা আগে ছাঁটুন — সাধারণত পুরোনো ইতিহাস (summarize করুন) বা কম প্রাসঙ্গিক chunk বাদ দিন। এভাবে ক্রম + প্রাসঙ্গিকতা + বাজেট — তিনটা মিলেই একটা অপটিমাইজড কন্টেক্সট তৈরি হয়।

মনে রাখুন

অপটিমাইজেশন মানে শুধু খরচ কমানো নয়। একটা পরিষ্কার, মনোযোগী কন্টেক্সট মডেলকে ভালো উত্তর দিতেও সাহায্য করে। সাশ্রয় আর মান — দুটো এখানে একসাথে আসে।

Licensed to sakilorb@gmail.com · Do not redistribute

কী শিখলেন

- ✓ মডেল শুরু ও শেষে বেশি মনোযোগ দেয় — তাই ক্রম গুরুত্বপূর্ণ।
- ✓ মূল নিয়ম উপরে, বর্তমান প্রশ্ন সবার শেষে রাখুন।
- ✓ প্রাসঙ্গিকতা > পরিমাণ — যা উত্তর বদলায় না, তা বাদ দিন।
- ✓ কন্টেক্সটকে বাজেট ধরে উপাদান অনুযায়ী টোকেন ভাগ করুন।
- ✓ আউটপুটের জন্য জায়গা রাখুন; বাজেট ছাড়ালে কম-গুরুত্বের অংশ আগে ছাঁটুন।

পরের অধ্যায়ে আমরা সব শেখা একসাথে কাজে লাগাব — শূন্য থেকে একটা পূর্ণ *ShopBondhu* চ্যাটবট গড়ব, ধাপে ধাপে।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১১

পূর্ণ কেস স্টাডি: ShopBondhu চ্যাটবট

এই অধ্যায়েই সব এক হবে। আমরা শূন্য থেকে একটা সম্পূর্ণ *ShopBondhu* সাপোর্ট চ্যাটবট গড়ব — বইয়ের প্রতিটা কৌশল কাজে লাগিয়ে: *system prompt, RAG, history, tools, output format* আর অপটিমাইজেশন।

১১.১ সমস্যা সংজ্ঞায়িত করা

কোড লেখার আগে স্পষ্ট করে নিই — বটটা ঠিক কী করবে আর কোথায় থামবে। অস্পষ্ট কাজের জন্য কন্টেক্সট ইঞ্জিনিয়ার করা যায় না; তাই পরিষ্কার স্পেসিফিকেশনই প্রথম ধাপ।

বটের দায়িত্ব ও সীমা

- ✓ **পণ্যের প্রশ্নের উত্তর** — ক্যাটালগ থেকে প্রাসঙ্গিক তথ্য নিয়ে (RAG)।
- ✓ **অর্ডার স্ট্যাটাস** — টুল দিয়ে আসল ডেটা এনে।
- ✓ **রিটার্ন/রিফান্ড** — শুধু পলিসির ভেতরে, সীমাসহ।
- ✓ **সবসময় বাংলায়**, **ভদ্রভাবে** — অনুমান না করে।
- ✓ **জটিল/রাগান্বিত কেস** — মানুষ-এজেন্টের কাছে হস্তান্তর।

১১.২ কন্টেক্সট পাইপলাইন ডিজাইন

কোড লেখার আগে নকশা। প্রতিটা টার্নে কন্টেক্সট কীভাবে তৈরি হবে, সেটা ম্যাপ করে নিই। লক্ষ্য করুন, প্রতিটা ধাপ আসলে বইয়ের একটা অধ্যায়:

প্রতি টার্নে কন্টেক্সট অ্যাসেম্বলি

- ৪ **System prompt** — ভূমিকা, নিয়ম, সীমা (অধ্যায় ৪)।
- ৭ **RAG** — প্রাসঙ্গিক পণ্য/পলিসি টেনে আনা (অধ্যায় ৭)।
- ৮ **History** — সংক্ষিপ্ত কথোপকথন ইতিহাস (অধ্যায় ৮)।

Licensed to sakilorb@gmail.com · Do not redistribute

৯ **Tools** — order_status, refund (অধ্যায় ৯)।

৬ **Output format** — reply + escalate ট্যাগ (অধ্যায় ৬)।

১০ **সাজানো ও বাজেট** — সঠিক ক্রম ও টোকেন সীমা (অধ্যায় ১০)।

১১.৩ ধাপে ধাপে কোড

এবার নকশা অনুযায়ী গড়ি। প্রথমে স্থায়ী system prompt আর টুলগুলো ঠিক করি:

```
SYSTEM_PROMPT = """তুমি ShopBondhu-র সাপোর্ট এজেন্ট "বন্ধু"।
- সবসময় ভদ্রভাবে বাংলায় উত্তর দাও।
- শুধু দেওয়া তথ্যের ভিত্তিতে উত্তর দাও, অনুমান করো না।
- এভাবে উত্তর দাও:
  <reply>ক্রেতার জন্য উত্তর</reply>
  <escalate>true/false</escalate>"""

TOOLS = [get_order_status, request_refund]   # অধ্যায় ৯
```

কোড ১১.১ — ভিত্তি: system prompt ও টুল।

এবার মূল ফাংশন — প্রতিটা ব্লক এক একটা অধ্যায় থেকে আসছে:

```
def respond(user_msg, history):
    # ১) RAG — প্রাসঙ্গিক ৩টি পণ্য/পলিসি (অধ্যায় ৭)
    hits = vector_db.search(embed(user_msg), top_k=3)
    data = "\n".join(h.as_text() for h in hits)

    # ২) সঠিক ক্রমে কন্টেক্সট সাজানো (অধ্যায় ১০)
    messages = (
        [{"role": "system", "content": SYSTEM_PROMPT}]     # নিয়ম উপরে
        + [{"role": "system", "content": f"তথ্য:\n{data}"}]
        + trim(history)                                    # সারাংশ (অধ্যায় ৮)
        + [{"role": "user", "content": user_msg}]          # প্রশ্ন শেষে
    )

    # ৩) টুলসহ মডেল কল (অধ্যায় ৯)
    out = call_model(messages, tools=TOOLS, on_tool=safe_handle)

    # ৪) আউটপুট পার্স ও সিদ্ধান্ত (অধ্যায় ৬)
    reply = extract_tag(out, "reply")
    if extract_tag(out, "escalate") == "true":
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
        route_to_human()
    return reply
```

কোড ১১.২ — পুরো পাইপলাইন: RAG → সাজানো → টুল → পার্স।

মাত্র কয়েক লাইনে গোটা বইটা ধরা পড়েছে। প্রতিটা অংশ আলাদা করে সহজ, কিন্তু একসাথে এরা একটা নির্ভরযোগ্য, বাস্তব সিস্টেম গড়ে।

১১.৪ সাধারণ বাগ ও সমাধান

বাস্তবে এমন সিস্টেম গড়তে গিয়ে কিছু সাধারণ সমস্যা প্রায়ই আসে। সাথে তাদের সমাধান:

সমস্যা → সমাধান

- **বট অর্ডার তথ্য বানিয়ে বলছে** → ডেটা আসলেই কন্টেক্সটে যাচ্ছে কিনা দেখুন; "অনুমান করো না" নিয়মটা জোরালো করুন (অধ্যায় ১, ৪)।
- **ফরম্যাট এলোমেলো** → few-shot উদাহরণ ও ট্যাগ/JSON ফরম্যাট যোগ করুন (অধ্যায় ৫, ৬)।
- **ধীর ও ব্যয়বহুল** → RAG top-k কমান, ইতিহাস summarize করুন, বাজেট মানুন (অধ্যায় ৩, ৮, ১০)।
- **ভুল পণ্য আসছে** → chunking ও embedding উন্নত করুন (অধ্যায় ৭)।
- **অনিয়মিত রিফান্ড প্রতিশ্রুতি** → সীমা ও টুল-গার্ড যোগ করুন (অধ্যায় ৪, ৯)।

মনে রাখুন

লক্ষ্য করুন, প্রতিটা বাগের সমাধানই বইয়ের কোনো না কোনো অধ্যায়ে আছে। এটাই context engineering-এর সৌন্দর্য — সমস্যাগুলো আলাদা মনে হলেও, সমাধান একই কাঠামোর ভেতরে।

সবশেষে দেখি বটটা বাস্তবে কেমন আচরণ করে। নিচে একটা নমুনা কথোপকথন — খেয়াল করুন কোথায় কোন কৌশল কাজ করছে:

নমুনা কথোপকথন

ক্রেতা ১৫ হাজারের মধ্যে ভালো হেডফোন আছে?

Licensed to sakilorb@gmail.com · Do not redistribute

বন্ধু: জি! SoundPro X2 (১২,৫০০৳) দারুণ একটা অপশন। আরও আছে BassBuds Pro (৯,৮০০৳)। কোনটির বিস্তারিত চান?

↑ *RAG থেকে প্রাসঙ্গিক পণ্য (অধ্যায় ৭)*

ক্রেতা: আমার আগের অর্ডার #SB-2043 কোথায়?

বন্ধু: আপনার #SB-2043 অর্ডারটি পথে আছে, ১৮ জুন ডেলিভারি হবে।

↑ *order_status টুল থেকে আসল ডেটা (অধ্যায় ৯)*

ক্রেতা: এটা একদম বাজে! আমি ম্যানেজারের সাথে কথা বলব!

বন্ধু: আপনার অসুবিধার জন্য আন্তরিক দুঃখিত। আমি বিষয়টি এখনই আমাদের টিমকে জানাচ্ছি, তাঁরা শিগগিরই যোগাযোগ করবেন।

↑ *escalate=true → মানুষের কাছে হস্তান্তর (অধ্যায় ৬)*

একটা কথোপকথনেই RAG, টুল আর escalation — তিনটাই কাজ করল। এটাই সব উপাদান একসাথে জোড়া লাগার ফল।

অনুশীলন

এবার তুমি বটটা একটু বাড়াও:

1. একটা নতুন টুল ভাবো — `track_complaint(order_id)` — যা অভিযোগের অবস্থা দেখায়। এর schema লেখো (অধ্যায় ৯)।
2. ক্রেতা যদি ইংরেজিতে প্রশ্ন করে, বট কী করবে? system prompt-এ একটা নিয়ম যোগ করো।
3. উপরের কথোপকথনে কোথায় context rot ঘটতে পারত, আর তুমি কীভাবে ঠেকাতে?

কী শিখলেন

- ✓ ভালো সিস্টেম শুরু হয় স্পষ্ট সমস্যা-সংজ্ঞা দিয়ে।
- ✓ কোডের আগে কন্টেক্সট পাইপলাইন নকশা করুন — প্রতিটা ধাপ একটা অধ্যায়।
- ✓ সব উপাদান (prompt, RAG, history, tools, format) একসাথে জোড়া লাগে।
- ✓ একটা পরিষ্কার পাইপলাইন কয়েক লাইনেই বাস্তব সিস্টেম গড়ে।

Licensed to sakilorb@gmail.com · Do not redistribute

✓ বেশিরভাগ বাগের সমাধান কোনো না কোনো মৌলিক কৌশলে ফিরে যায়।

শেষ অধ্যায়ে আমরা একটা সংক্ষিপ্ত সেরা-অভ্যাসের চেকলিস্ট দেখব — যা যেকোনো *AI* প্রজেক্টে কাজে লাগানোর আগে একবার মিলিয়ে নিতে পারবেন।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১২

সাধারণ ভুল ও সেরা অভ্যাস

পুরো বইয়ের শিক্ষা এবার এক জায়গায় গুছিয়ে নিই। এই অধ্যায়টা একটা রেফারেন্স — যেকোনো *AI* প্রজেক্ট শুরু বা শিপ করার আগে একবার চোখ বুলিয়ে নেওয়ার মতো।

সবচেয়ে সাধারণ ভুলগুলো

নতুন ডেভেলপাররা প্রায়ই কয়েকটা একই ভুল করেন। আগেভাগে চিনে রাখলে অনেক সময় বাঁচে:

এড়িয়ে চলুন

- ✗ **ডেটা ছাড়াই উত্তর আশা করা** — সুন্দর প্রম্পট লিখেও প্রাসঙ্গিক তথ্য না দেওয়া।
- ✗ **"নিরাপত্তার জন্য" সব ঢেলে দেওয়া** — যা context rot ও খরচ বাড়ায়।
- ✗ **মডেলকে অন্ধভাবে বিশ্বাস করা** — আউটপুট যাচাই না করে ব্যবহার করা।
- ✗ **অস্পষ্ট নির্দেশনা** — "ভালো হও" ধরনের, মাপা নয় এমন নিয়ম।
- ✗ **সীমাহীন টুল দেওয়া** — সংবেদনশীল কাজে গার্ড না রাখা।

সেরা অভ্যাসের চেকলিস্ট

আর এই অভ্যাসগুলো ধরে রাখলে আপনার সিস্টেম নির্ভরযোগ্য, সাশ্রয়ী আর রক্ষণাবেক্ষণযোগ্য থাকবে:

▸ কন্টেক্সট ডিজাইন

- ☑ **System prompt**-এ পরিষ্কার ভূমিকা, নিয়ম ও সীমা আছে।
- ☑ নির্দেশনা ইতিবাচক ও সুনির্দিষ্ট ("৩-৪ বাক্যে", "ভদ্রভাবে")।
- ☑ মূল নিয়ম উপরে, বর্তমান প্রশ্ন সবার শেষে রাখা হয়েছে।

▸ ডেটা ও RAG

- ☑ শুধু **প্রাসঙ্গিক** তথ্য আসছে, পুরো ভাণ্ডার নয়।
- ☑ Chunking অর্থপূর্ণ — খুব বড়ও নয়, খুব ছোটও নয়।
- ☑ Retrieval-এর মান যাচাই করা হয়েছে।

Licensed to sakilorb@gmail.com · Do not redistribute

▸ ফরম্যাট ও নিরাপত্তা

- ☑ আউটপুট কাঠামোবদ্ধ (JSON/ট্যাগ) ও নিরাপদে পার্স করা হয়।
- ☑ টুলের আর্গুমেন্ট যাচাই ও অনুমোদন আছে।
- ☑ সংবেদনশীল কাজে সীমা ও মানুষ-ইন-দ্য-লুপ আছে।

▸ অপটিমাইজেশন

- ☑ ইতিহাস summarize করে ছোট রাখা হয়।
- ☑ টোকেন বাজেট ঠিক করা, আউটপুটের জন্য জায়গা রাখা।
- ☑ বাস্তব প্রশ্ন দিয়ে পরীক্ষা ও ইটারেট করা হয়েছে।

মনে রাখুন

এই চেকলিস্টটা প্রজেক্ট শিপ করার আগে একবার মিলিয়ে নিন। বেশিরভাগ প্রোডাকশন সমস্যা এই তালিকার কোনো একটা বাদ পড়া আইটেম থেকেই আসে।

কী শিখলেন

- ✓ সাধারণ ভুলগুলো আগেভাগে চিনলে অনেক সময় ও খরচ বাঁচে।
- ✓ ভালো কন্টেক্সট ডিজাইন শুরু হয় পরিষ্কার নিয়ম ও সঠিক ক্রম দিয়ে।
- ✓ প্রাসঙ্গিকতা, নিরাপদ ফরম্যাট ও টুল-গার্ড নির্ভরযোগ্যতার ভিত্তি।
- ✓ শিপ করার আগে একটা চেকলিস্ট মিলিয়ে নেওয়া পেশাদারি অভ্যাস।

Licensed to sakilorb@gmail.com · Do not redistribute

শেষ কথা

অভিনন্দন — আপনি পুরো যাত্রাটা শেষ করলেন! আমরা শুরু করেছিলাম একটা সহজ প্রশ্ন দিয়ে: "কন্টেক্সট আসলে কী?" আর শেষ করলাম শূন্য থেকে একটা পূর্ণ চ্যাটবট গড়ে।

মনে রাখার মতো একটাই কথা যদি বেছে নিতে হয়, সেটা হলো — **ভালো AI অ্যাপ মানে বড় মডেল নয়, ভালোভাবে সাজানো কন্টেক্সট।** মডেল একজন মেধাবী সহকারী; আপনার কাজ তাকে সঠিক তথ্য, সঠিক সময়ে, সঠিক ক্রমে তুলে দেওয়া।

এখন আসল শেখা শুরু — নিজের একটা ছোট প্রজেক্ট নিন, আর এই কৌশলগুলো হাতে-কলমে প্রয়োগ করুন। ভুল হবে, আর সেই ভুল থেকেই দক্ষতা আসবে। শুভকামনা।

— ইফতে সামউল ওহী

iftesamulohy.com

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট ক

Context Engineering চিটশিট

পুরো বইয়ের মূল কথাগুলো এক নজরে — কাজের সময় দ্রুত মিলিয়ে নেওয়ার জন্য।

কন্টেক্সট

মডেলের নিজস্ব স্মৃতি নেই; context window-এ যা রাখবেন, শুধু তাই সে জানে।

Prompt vs Context

প্রম্পট = একটা নির্দেশনা; Context = পুরো পরিবেশ (নিয়ম+ডেটা+উদাহরণ+টুল)।

টোকেন

ইনপুট ও আউটপুট দুটোই গোনা হয়; বাংলা বেশি খরচ করে। প্রাসঙ্গিকতা > পরিমাণ।

System Prompt

পরিষ্কার ভূমিকা + সুনির্দিষ্ট নিয়ম + সীমা; ইতিবাচক ভাষায় লিখুন।

Few-shot

২-৫টি ভারসাম্যপূর্ণ উদাহরণ দিন; মডেল ফরম্যাট হুবহু কপি করে।

Output Format

JSON বা ট্যাগ চান; সবসময় নিরাপদে পার্স ও ভ্যালিডেট করুন।

RAG

প্রাসঙ্গিক chunk টেনে আনুন, পুরো ভাণ্ডার নয়; retrieval-এর মানই সব।

Memory

ইতিহাস আবার পাঠান; বড় হলে summarize; দীর্ঘমেয়াদি = store + retrieval।

Tools

মডেল চায়, আপনার কোড চালায়; যাচাই + সীমা + মানুষ-ইন-দ্য-লুপ।

অপটিমাইজেশন

ক্রম (নিয়ম উপরে, প্রশ্ন শেষে) + টোকেন বাজেট + প্রাসঙ্গিকতা।

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট খ

শব্দকোষ

বইয়ে ব্যবহৃত টেকনিক্যাল টার্মগুলোর সংক্ষিপ্ত বাংলা অর্থ।

Token (টোকেন)
লেখার ছোট টুকরো; মডেল এই এককেই লেখা পড়ে ও গোনে।

Context Window
মডেলের "ডেস্ক" — একবারে যতটুকু তথ্য নিতে পারে তার সীমা।

Prompt Engineering
মডেলকে দেওয়া নির্দেশনা/প্রশ্ন ভালোভাবে লেখার কৌশল।

Context Engineering
মডেলের চারপাশের পুরো পরিবেশ (তথ্য, উদাহরণ, টুল, ফরম্যাট) ডিজাইন করা।

System Prompt
পুরো কথোপকথনজুড়ে প্রযোজ্য স্থায়ী ভূমিকা ও নিয়মের ব্রিফ।

Few-shot / Zero-shot
কয়েকটা উদাহরণ দিয়ে / উদাহরণ ছাড়া মডেলকে কাজ করানো।

Hallucination
মডেলের অনুমান করে ভুল বা বানানো তথ্য দেওয়া।

RAG
Retrieval-Augmented Generation — প্রাসঙ্গিক তথ্য খুঁজে এনে উত্তর তৈরি।

Embedding
লেখাকে অর্থবাহী সংখ্যা-ভেক্টরে রূপান্তর; কাছের অর্থ = কাছের ভেক্টর।

Vector Search
embedding-এর মিল দেখে প্রাসঙ্গিক তথ্য খুঁজে বের করা।

Chunking
বড় লেখাকে অর্থপূর্ণ ছোট টুকরোয় ভাগ করা।

Context Rot
বেশি অপ্রাসঙ্গিক তথ্যের কারণে উত্তরের মান কমে যাওয়া।

Licensed to sakilorb@gmail.com · Do not redistribute

Function Calling / Tools

মডেলকে বাস্তব কাজ (যেমন DB কল) ট্রিগার করার ক্ষমতা দেওয়া।

Summarization

পুরোনো কথোপকথন ছোট সারাংশে চেপে ইতিহাস হালকা রাখা।

Escalation

জটিল/সংবেদনশীল কেস মানুষ-এজেন্টের কাছে হস্তান্তর করা।

Latency

প্রশ্ন থেকে উত্তর পাওয়া পর্যন্ত লাগা সময়।

Licensed to sakilorb@gmail.com · Do not redistribute

পরিশিষ্ট গ

আরও শেখার রিসোর্স

এই বই একটা শুরু। আরও গভীরে যেতে চাইলে নিচের দিকগুলো কাজে লাগবে।

অফিসিয়াল ডকুমেন্টেশন
আপনি যে মডেল ব্যবহার করছেন তার অফিসিয়াল ডকস (যেমন Anthropic, OpenAI) — টুল, JSON মোড ও সর্বশেষ ফিচার সেখানেই নির্ভুলভাবে পাবেন।

Vector Database ও Embedding
RAG হাতে-কলমে শিখতে FAISS, pgvector বা Pinecone-এর মতো টুল নিয়ে ছোট একটা প্রজেক্ট করুন।

নিজের প্রজেক্ট গড়ুন
সবচেয়ে ভালো শেখা হাতে-কলমে। একটা ছোট চ্যাটবট বা টুল বানান, আর বইয়ের চেকলিস্ট ধরে এগোন।

লেখকের কোর্স
Python, Django ও AI অটোমেশন নিয়ে আরও শিখতে — iftesamulohy.com।

শেষ পরামর্শ

পড়া যথেষ্ট নয় — আজই একটা ছোট কিছু বানান। এক সপ্তাহ হাতে-কলমে কাজ করলে এই বইয়ের কৌশলগুলো আপনার নিজের হয়ে যাবে।

Licensed to sakilorb@gmail.com · Do not redistribute